

```
In [44]: import os
import cv2
import torch
import torch.nn as nn
import torch.optim as optim
from torch.nn.utils.rnn import pad_sequence, pack_padded_sequence
import numpy as np
import matplotlib.pyplot as plt
from collections import deque
from scipy.fft import fft
from scipy.signal import find_peaks
from scipy.spatial import Delaunay
import mediapipe as mp
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import confusion_matrix
from sklearn.utils.class_weight import compute_class_weight
import seaborn as sns
```

```

In [45]: # Initialize MediaPipe
mp_face_mesh = mp.solutions.face_mesh
face_mesh = mp_face_mesh.FaceMesh(
    static_image_mode=False,
    max_num_faces=1,
    refine_landmarks=True,
    min_detection_confidence=0.3,
    min_tracking_confidence=0.5
)
mp_drawing = mp.solutions.drawing_utils

# --- Data Augmentation Functions ---
def add_temporal_noise(seq, noise_level=0.02):
    """
    Add small random Gaussian noise to feature sequences for robustness.
    seq: sequence of shape (T, F) where T=frames, F=features
    noise_level: standard deviation of noise
    """
    noise = np.random.normal(0, noise_level, seq.shape)
    return seq + noise

def temporal_dropout(seq, dropout_rate=0.1):
    """
    Randomly drop frames from sequence to simulate varying frame rates.
    seq: sequence of shape (T, F)
    dropout_rate: probability of dropping a frame
    """
    if np.random.rand() < 0.3: # 30% chance to apply dropout
        mask = np.random.rand(len(seq)) > dropout_rate
        if np.sum(mask) > 1: # Keep at least 2 frames
            return seq[mask]
    return seq

def normalize_for_rotation_distance(landmarks, prev_landmarks):
    """
    Normalize landmarks for camera distance and face rotation.
    landmarks: list of (x, y, z) tuples
    """
    if landmarks is None or len(landmarks) == 0:
        return landmarks

    # Use nose tip as reference
    nose_tip = np.array(landmarks[1])

    # Normalize by distance from nose
    normalized = []
    for lm in landmarks:
        lm_arr = np.array(lm)
        # Distance-invariant: divide by distance from nose
        dist = np.linalg.norm(lm_arr - nose_tip) + 1e-6
        normalized.append((lm_arr - nose_tip) / dist)

```

```
return normalized
```

```
In [46]: # --- Landmark Indices ---
left_brow_idx = [70, 63, 105, 66, 107, 55, 65, 52, 53, 46]
right_brow_idx = [300, 293, 334, 296, 336, 285, 295, 282, 283, 276]
inner_brow_idx = [63, 293]
brow_landmarks_idx = list(set(left_brow_idx + right_brow_idx +
inner_brow_idx))

left_cheek_idx = [205, 206, 216, 204, 207, 114, 115, 116, 213, 214, 215]
right_cheek_idx = [425, 426, 436, 424, 427, 343, 344, 345, 433, 434, 435]
cheek_landmarks_idx = list(set(left_cheek_idx + right_cheek_idx))

left_eye_idx = [33, 160, 158, 133, 153, 144, 145, 159]
right_eye_idx = [362, 385, 387, 263, 373, 374, 380, 386]
outer_eye_idx = [33, 133, 362, 263]
eye_landmarks_idx = list(set(left_eye_idx + right_eye_idx + outer_eye_idx))

jaw_landmarks_idx = [152, 176, 136, 172, 397, 365, 366, 379, 400, 378, 377]

lip_landmarks_idx = [13, 14, 37, 39, 40, 61, 78, 80, 81, 82, 84, 91, 95,
146, 178, 181, 185, 191, 267, 269, 270, 291, 308, 310, 311, 312, 314, 321,
324, 375, 402, 405, 409, 415]

mouth_landmarks_idx = [13, 14, 17, 37, 39, 40, 61, 78, 80, 81, 82, 84, 87,
88, 91, 95, 146, 178, 181, 185, 191, 267, 269, 270, 291, 308, 310, 311,
312, 314, 317, 318, 321, 324, 375, 402, 405, 409, 415]

# --- LEFT / RIGHT Surface Vector Splits ---
left_brow_idx_surface = [70, 63, 105, 66, 107, 55, 65, 52, 53, 46]
right_brow_idx_surface = [300, 293, 334, 296, 336, 285, 295, 282, 283, 276]

left_cheek_idx_surface = [205, 206, 216, 204, 207, 114, 115, 116]
right_cheek_idx_surface = [425, 426, 436, 424, 427, 343, 344, 345]

left_eye_idx_surface = [33, 160, 158, 133, 153, 144, 145, 159]
right_eye_idx_surface = [362, 385, 387, 263, 373, 374, 380, 386]

left_jaw_idx_surface = [152, 176, 136, 172]
right_jaw_idx_surface = [397, 365, 366, 379, 400, 378, 377]

left_lip_idx_surface = [61, 78, 80, 81, 82, 84, 91, 95]
right_lip_idx_surface = [291, 308, 310, 311, 312, 314, 321, 324]

left_mouth_idx_surface = [61, 78, 80, 81, 82, 84, 87, 88, 91, 95, 17]
right_mouth_idx_surface = [291, 308, 310, 311, 312, 314, 317, 318, 321,
324, 375]
```

In [47]:

```
# --- Buffers ---
buffer_size = 10
brow_raise_buffer = deque(maxlen=buffer_size)
brow_left_raise_buffer = deque(maxlen=buffer_size)
brow_right_raise_buffer = deque(maxlen=buffer_size)
brow_inner_raise_buffer = deque(maxlen=buffer_size)
brow_vel_buffer = deque(maxlen=buffer_size)
brow_surface_var_buffer = deque(maxlen=buffer_size)
brow_surface_dir_buffer = deque(maxlen=buffer_size)

cheek_raise_buffer = deque(maxlen=buffer_size)
cheek_vel_buffer = deque(maxlen=buffer_size)
cheek_surface_var_buffer = deque(maxlen=buffer_size)
cheek_surface_dir_buffer = deque(maxlen=buffer_size)

eye_ratio_buffer = deque(maxlen=buffer_size)
eye_vel_buffer = deque(maxlen=buffer_size)
blink_buffer = deque(maxlen=30)
eye_surface_var_buffer = deque(maxlen=buffer_size)
eye_surface_dir_buffer = deque(maxlen=buffer_size)

jaw_open_buffer = deque(maxlen=buffer_size)
jaw_vel_buffer = deque(maxlen=buffer_size)
jaw_surface_var_buffer = deque(maxlen=buffer_size)
jaw_surface_dir_buffer = deque(maxlen=buffer_size)

lips_open_buffer = deque(maxlen=buffer_size)
lips_vel_buffer = deque(maxlen=buffer_size)
lips_surface_var_buffer = deque(maxlen=buffer_size)
lips_surface_dir_buffer = deque(maxlen=buffer_size)

mouth_open_buffer = deque(maxlen=buffer_size)
mouth_vel_buffer = deque(maxlen=buffer_size)
mouth_surface_var_buffer = deque(maxlen=buffer_size)
mouth_surface_dir_buffer = deque(maxlen=buffer_size)
```

In [48]:

```
# --- Global ---
frame_global = None
features_global = {
    'Eye': {}, 'Brow': {}, 'Cheek': {}, 'Mouth': {}, 'Lips': {}, 'Jaw': {}
}
prev_landmarks_global = None
```



```

In [49]: # --- Surface Vector Split Function ---
def compute_surface_vectors_split(landmarks, prev_landmarks, left_idx,
right_idx):
    if prev_landmarks is None or landmarks is None:
        zero = {'vectors': np.array([]), 'positions': [], 'norms':
np.array([]), 'mean_mag': 0.0, 'var': 0.0, 'angle': 0.0}
        return {'left': zero, 'right': zero}

    def process_side(idx_list):
        if not idx_list:
            return {'vectors': np.array([]), 'positions': [], 'norms':
np.array([]), 'mean_mag': 0.0, 'var': 0.0, 'angle': 0.0}

        curr_pos = []
        prev_pos = []
        for idx in idx_list:
            if idx >= len(landmarks) or idx >= len(prev_landmarks):
                continue
            curr_pos.append(np.array(landmarks[idx]))
            prev_pos.append(np.array(prev_landmarks[idx]))

        if len(curr_pos) < 3:
            return {'vectors': np.array([]), 'positions': [], 'norms':
np.array([]), 'mean_mag': 0.0, 'var': 0.0, 'angle': 0.0}

        points2d = np.array([p[:2] for p in curr_pos])

        try:
            tri = Delaunay(points2d)
        except:
            return {'vectors': np.array([]), 'positions': [], 'norms':
np.array([]), 'mean_mag': 0.0, 'var': 0.0, 'angle': 0.0}

        triangle_norms = []
        triangle_vectors = []
        triangle_areas = []

        for simplex in tri.simplices:
            i1, i2, i3 = simplex
            v1 = curr_pos[i1] - prev_pos[i1]
            v2 = curr_pos[i2] - prev_pos[i2]
            v3 = curr_pos[i3] - prev_pos[i3]
            mean_v = (v1 + v2 + v3) / 3
            norm = np.linalg.norm(mean_v)
            triangle_norms.append(norm)
            if norm > 1e-6:
                triangle_vectors.append(mean_v / norm)
            else:
                triangle_vectors.append(mean_v)
            area = 0.5 * np.abs(np.cross(points2d[i2] - points2d[i1],
points2d[i3] - points2d[i1]))
            triangle_areas.append(area)

```

```

triangle_norms = np.array(triangle_norms)
mean_mag = np.mean(triangle_norms) if len(triangle_norms) > 0 else
0.0

var = np.var(triangle_norms) if len(triangle_norms) > 0 else 0.0

angle = 0.0
if triangle_vectors and triangle_areas:
    weighted_vectors = np.array(triangle_vectors) *
np.array(triangle_areas)[:, np.newaxis]
    avg = np.sum(weighted_vectors[:, :2], axis=0) /
np.sum(triangle_areas)
    n = np.linalg.norm(avg)
    if n > 1e-6:
        angle = np.arctan2(avg[1], avg[0])

    vectors = np.array(triangle_vectors) if triangle_vectors else
np.array([])
    norms = triangle_norms
    positions = [np.mean([curr_pos[i] for i in s], axis=0) for s in
tri.simplices]

    return {'vectors': vectors, 'positions': positions, 'norms': norms,
'mean_mag': mean_mag, 'var': var, 'angle': angle}

    return {'left': process_side(left_idx), 'right':
process_side(right_idx)}

```

```

In [51]: # --- BROW ---
def compute_brow_features(landmarks, prev_landmarks):
    if landmarks is None: return {}
    nose_tip = np.array(landmarks[1])
    norm_landmarks = [np.array(lm) - nose_tip for lm in landmarks]

    left_brow_ys = [norm_landmarks[i][1] for i in left_brow_idx]
    right_brow_ys = [norm_landmarks[i][1] for i in right_brow_idx]
    inner_brow_ys = [norm_landmarks[i][1] for i in inner_brow_idx]

    left_raise = -np.mean(left_brow_ys)
    right_raise = -np.mean(right_brow_ys)
    inner_raise = -np.mean(inner_brow_ys)
    overall_raise = (left_raise + right_raise) / 2

    brow_left_raise_buffer.append(left_raise)
    brow_right_raise_buffer.append(right_raise)
    brow_inner_raise_buffer.append(inner_raise)
    brow_raise_buffer.append(overall_raise)

    left_stats = [np.mean(brow_left_raise_buffer),
np.std(brow_left_raise_buffer)] if len(brow_left_raise_buffer) > 1 else [0,
0]
    right_stats = [np.mean(brow_right_raise_buffer),
np.std(brow_right_raise_buffer)] if len(brow_right_raise_buffer) > 1 else
[0, 0]
    inner_stats = [np.mean(brow_inner_raise_buffer),
np.std(brow_inner_raise_buffer)] if len(brow_inner_raise_buffer) > 1 else
[0, 0]

    brow_vel = abs(overall_raise - brow_raise_buffer[-2]) if
len(brow_raise_buffer) > 1 else 0
    brow_vel_buffer.append(brow_vel)
    vel_stats = [np.mean(brow_vel_buffer), np.std(brow_vel_buffer)] if
len(brow_vel_buffer) > 1 else [0, 0]
    rapid_count = len(find_peaks(list(brow_vel_buffer), distance=2)[0]) if
len(brow_vel_buffer) > 1 else 0

    micro_var = np.var(brow_raise_buffer) if len(brow_raise_buffer) > 1
else 0.0
    freq_mean = np.mean(np.abs(fft(list(brow_raise_buffer))))
[:buffer_size//2]) if len(brow_raise_buffer) == buffer_size else 0.0
    peak_freq = np.max(np.abs(fft(list(brow_raise_buffer))))
[:buffer_size//2]) if len(brow_raise_buffer) == buffer_size else 0.0

    brow_asym = abs(left_raise - right_raise)
    inner_asym = abs(norm_landmarks[63][1] - norm_landmarks[293][1])
    asym_diffs = np.abs(np.array(brow_left_raise_buffer) -
np.array(brow_right_raise_buffer))
    temp_asym_var = np.var(asym_diffs) if len(asym_diffs) > 1 else 0.0

    surface = compute_surface_vectors_split(landmarks, prev_landmarks,
left_brow_idx_surface, right_brow_idx_surface)

```

```

left, right = surface['left'], surface['right']

brow_surface_var_buffer.append({'left': left['var'], 'right':
right['var']})
brow_surface_dir_buffer.append({'left': left['angle'], 'right':
right['angle']})

left_vars = [x['left'] for x in list(brow_surface_var_buffer)[-10:]]
right_vars = [x['right'] for x in list(brow_surface_var_buffer)[-10:]]
left_angles = [x['left'] for x in list(brow_surface_dir_buffer)[-10:]]
right_angles = [x['right'] for x in list(brow_surface_dir_buffer)
[-10:]]

var_stats_l = [np.mean(left_vars), np.std(left_vars),
np.min(left_vars), np.max(left_vars)] if left_vars else [0]*4
var_stats_r = [np.mean(right_vars), np.std(right_vars),
np.min(right_vars), np.max(right_vars)] if right_vars else [0]*4
dir_stats_l = [np.mean(left_angles), np.std(left_angles)] if
len(left_angles) > 1 else [0, 0]
dir_stats_r = [np.mean(right_angles), np.std(right_angles)] if
len(right_angles) > 1 else [0, 0]

return {
    'Brow micro-expression variance mean': micro_var,
    'Brow micro-expression rapid changes count': rapid_count,
    'Brow velocity (mean)': vel_stats[0],
    'Brow velocity (std)': vel_stats[1],
    'Right brow raise (mean)': right_stats[0],
    'Right brow raise (std)': right_stats[1],
    'Left brow raise (mean)': left_stats[0],
    'Left brow raise (std)': left_stats[1],
    'Inner brow raise (mean)': inner_stats[0],
    'Inner brow raise (std)': inner_stats[1],
    'Brow asymmetry (mean)': brow_asym,
    'Temporal brow asymmetry variance': temp_asym_var,
    'Brow frequency mean': freq_mean,
    'Brow peak frequency': peak_freq,

    'Left surface vector magnitude mean': left['mean_mag'],
    'Left surface variance (current)': left['var'],
    'Left surface variance mean': var_stats_l[0],
    'Left surface variance std': var_stats_l[1],
    'Left surface variance min': var_stats_l[2],
    'Left surface variance max': var_stats_l[3],
    'Left surface dominant angle mean': dir_stats_l[0],
    'Left surface dominant angle std': dir_stats_l[1],

    'Right surface vector magnitude mean': right['mean_mag'],
    'Right surface variance (current)': right['var'],
    'Right surface variance mean': var_stats_r[0],
    'Right surface variance std': var_stats_r[1],
    'Right surface variance min': var_stats_r[2],
    'Right surface variance max': var_stats_r[3],

```

```

        'Right surface dominant angle mean': dir_stats_r[0],
        'Right surface dominant angle std': dir_stats_r[1],
    }

# --- CHEEK ---
def compute_cheek_features(landmarks, prev_landmarks):
    if landmarks is None: return {}
    nose_tip = np.array(landmarks[1])
    norm_landmarks = [np.array(lm) - nose_tip for lm in landmarks]

    left_ys = [norm_landmarks[i][1] for i in left_cheek_idx if i <
len(norm_landmarks)]
    left_xs = [norm_landmarks[i][0] for i in left_cheek_idx if i <
len(norm_landmarks)]
    right_ys = [norm_landmarks[i][1] for i in right_cheek_idx if i <
len(norm_landmarks)]
    right_xs = [norm_landmarks[i][0] for i in right_cheek_idx if i <
len(norm_landmarks)]

    left_raise = -np.mean(left_ys) + np.mean(left_xs) if left_ys else 0
    right_raise = -np.mean(right_ys) + np.mean(right_xs) if right_ys else 0
    overall = (left_raise + right_raise) / 2

    cheek_raise_buffer.append(overall)
    stats = [np.mean(cheek_raise_buffer), np.std(cheek_raise_buffer)] if
len(cheek_raise_buffer) > 1 else [0, 0]
    vel = abs(overall - cheek_raise_buffer[-2]) if len(cheek_raise_buffer)
> 1 else 0
    cheek_vel_buffer.append(vel)
    vel_stats = [np.mean(cheek_vel_buffer), np.std(cheek_vel_buffer)] if
len(cheek_vel_buffer) > 1 else [0, 0]
    rapid = len(find_peaks(list(cheek_vel_buffer), distance=2)[0]) if
len(cheek_vel_buffer) > 1 else 0
    var = np.var(cheek_raise_buffer) if len(cheek_raise_buffer) > 1 else 0
    freq = np.mean(np.abs(fft(list(cheek_raise_buffer))[:buffer_size//2]))
if len(cheek_raise_buffer) == buffer_size else 0

    asym = abs(left_raise - right_raise)
    surface = compute_surface_vectors_split(landmarks, prev_landmarks,
left_cheek_idx_surface, right_cheek_idx_surface)
    l, r = surface['left'], surface['right']
    cheek_surface_var_buffer.append({'left': l['var'], 'right': r['var']})
    cheek_surface_dir_buffer.append({'left': l['angle'], 'right':
r['angle']})

    lv = [x['left'] for x in list(cheek_surface_var_buffer)[-10:]]
    rv = [x['right'] for x in list(cheek_surface_var_buffer)[-10:]]
    la = [x['left'] for x in list(cheek_surface_dir_buffer)[-10:]]
    ra = [x['right'] for x in list(cheek_surface_dir_buffer)[-10:]]

    vl = [np.mean(lv), np.std(lv), np.min(lv), np.max(lv)] if lv else [0]*4
    vr = [np.mean(rv), np.std(rv), np.min(rv), np.max(rv)] if rv else [0]*4

```

```

dl = [np.mean(la), np.std(la)] if len(la) > 1 else [0, 0]
dr = [np.mean(ra), np.std(ra)] if len(ra) > 1 else [0, 0]

return {
    'Cheek puff micro-expression variance mean': var,
    'Cheek puff rapid changes count': rapid,
    'Cheek raise (mean)': stats[0],
    'Cheek raise (std)': stats[1],
    'Cheek velocity (mean)': vel_stats[0],
    'Cheek velocity (std)': vel_stats[1],
    'Cheek frequency mean': freq,
    'Cheek asymmetry (mean)': asym,

    'Left surface vector magnitude mean': l['mean_mag'],
    'Left surface variance (current)': l['var'],
    'Left surface variance mean': vl[0],
    'Left surface variance std': vl[1],
    'Left surface variance min': vl[2],
    'Left surface variance max': vl[3],
    'Left surface dominant angle mean': dl[0],
    'Left surface dominant angle std': dl[1],

    'Right surface vector magnitude mean': r['mean_mag'],
    'Right surface variance (current)': r['var'],
    'Right surface variance mean': vr[0],
    'Right surface variance std': vr[1],
    'Right surface variance min': vr[2],
    'Right surface variance max': vr[3],
    'Right surface dominant angle mean': dr[0],
    'Right surface dominant angle std': dr[1],
}

# --- EYE ---
def compute_eye_features(landmarks, prev_landmarks):
    if landmarks is None: return {}
    nose_tip = np.array(landmarks[1])
    norm_landmarks = [np.array(lm) - nose_tip for lm in landmarks]

    lu = (norm_landmarks[159][1] + norm_landmarks[158][1] +
norm_landmarks[160][1]) / 3
    ll = (norm_landmarks[145][1] + norm_landmarks[144][1] +
norm_landmarks[153][1]) / 3
    lw = abs(norm_landmarks[33][0] - norm_landmarks[133][0])
    left_ratio = abs(lu - ll) / lw if lw > 0 else 0

    ru = (norm_landmarks[386][1] + norm_landmarks[387][1] +
norm_landmarks[385][1]) / 3
    rl = (norm_landmarks[374][1] + norm_landmarks[373][1] +
norm_landmarks[380][1]) / 3
    rw = abs(norm_landmarks[362][0] - norm_landmarks[263][0])
    right_ratio = abs(ru - rl) / rw if rw > 0 else 0

    ratio = (left_ratio + right_ratio) / 2

```

```

    eye_ratio_buffer.append(ratio)
    stats = [np.mean(eye_ratio_buffer), np.std(eye_ratio_buffer)] if
len(eye_ratio_buffer) > 1 else [0, 0]
    vel = abs(ratio - eye_ratio_buffer[-2]) if len(eye_ratio_buffer) > 1
else 0
    eye_vel_buffer.append(vel)
    vel_stats = [np.mean(eye_vel_buffer), np.std(eye_vel_buffer)] if
len(eye_vel_buffer) > 1 else [0, 0]
    rapid = len(find_peaks(list(eye_vel_buffer), distance=2)[0]) if
len(eye_vel_buffer) > 1 else 0
    var = np.var(eye_ratio_buffer) if len(eye_ratio_buffer) > 1 else 0
    blink = 1 if ratio < 0.1 else 0
    blink_buffer.append(blink)
    blink_rate = sum(blink_buffer) / len(blink_buffer) if blink_buffer else
0

    surface = compute_surface_vectors_split(landmarks, prev_landmarks,
left_eye_idx_surface, right_eye_idx_surface)
    l, r = surface['left'], surface['right']
    eye_surface_var_buffer.append({'left': l['var'], 'right': r['var']})
    eye_surface_dir_buffer.append({'left': l['angle'], 'right':
r['angle']})

    lv = [x['left'] for x in list(eye_surface_var_buffer)[-10:]]
    rv = [x['right'] for x in list(eye_surface_var_buffer)[-10:]]
    la = [x['left'] for x in list(eye_surface_dir_buffer)[-10:]]
    ra = [x['right'] for x in list(eye_surface_dir_buffer)[-10:]]

    vl = [np.mean(lv), np.std(lv), np.min(lv), np.max(lv)] if lv else [0]*4
    vr = [np.mean(rv), np.std(rv), np.min(rv), np.max(rv)] if rv else [0]*4
    dl = [np.mean(la), np.std(la)] if len(la) > 1 else [0, 0]
    dr = [np.mean(ra), np.std(ra)] if len(ra) > 1 else [0, 0]

    return {
        'Eye widening micro-expression variance mean': var,
        'Eye widening rapid changes count': rapid,
        'Eye ratio (mean)': stats[0],
        'Eye ratio (std)': stats[1],
        'Blink rate': blink_rate,
        'Eye squint velocity (mean)': vel_stats[0],
        'Eye squint velocity (std)': vel_stats[1],

        'Left surface vector magnitude mean': l['mean_mag'],
        'Left surface variance (current)': l['var'],
        'Left surface variance mean': vl[0],
        'Left surface variance std': vl[1],
        'Left surface variance min': vl[2],
        'Left surface variance max': vl[3],
        'Left surface dominant angle mean': dl[0],
        'Left surface dominant angle std': dl[1],

        'Right surface vector magnitude mean': r['mean_mag'],
        'Right surface variance (current)': r['var'],

```

```

        'Right surface variance mean': vr[0],
        'Right surface variance std': vr[1],
        'Right surface variance min': vr[2],
        'Right surface variance max': vr[3],
        'Right surface dominant angle mean': dr[0],
        'Right surface dominant angle std': dr[1],
    }

# --- JAW ---
def compute_jaw_features(landmarks, prev_landmarks):
    if landmarks is None: return {}
    nose_tip = np.array(landmarks[1])
    norm_landmarks = [np.array(lm) - nose_tip for lm in landmarks]

    chin = norm_landmarks[152]
    upper_jaw_ref = norm_landmarks[13]
    jaw_open = np.linalg.norm(chin - upper_jaw_ref)
    jaw_open_buffer.append(jaw_open)
    jaw_open_stats = [np.mean(jaw_open_buffer), np.std(jaw_open_buffer),
np.min(jaw_open_buffer), np.max(jaw_open_buffer)] if len(jaw_open_buffer) >
1 else [0.0, 0.0, 0.0, 0.0]

    jaw_vel = abs(jaw_open - jaw_open_buffer[-2]) if len(jaw_open_buffer) >
1 else 0
    jaw_vel_buffer.append(jaw_vel)
    jaw_vel_stats = [np.mean(jaw_vel_buffer), np.std(jaw_vel_buffer)] if
len(jaw_vel_buffer) > 1 else [0.0, 0.0]

    left_jaw = norm_landmarks[136]
    right_jaw = norm_landmarks[400]
    jaw_asym = np.abs(left_jaw[0] - right_jaw[0])
    jaw_asym_stats = [np.mean([jaw_asym]), np.std([jaw_asym]),
np.max([jaw_asym])] if len(jaw_open_buffer) > 1 else [0.0, 0.0, 0.0]

    rapid_count = len(find_peaks(list(jaw_vel_buffer), distance=2)[0]) if
len(jaw_vel_buffer) > 1 else 0
    sig_mov_count = sum(1 for v in jaw_vel_buffer if v > 0.001)

    freq_mean = np.mean(np.abs(fft(list(jaw_open_buffer)))
[:buffer_size//2]) if len(jaw_open_buffer) == buffer_size else 0.0
    peak_freq = np.max(np.abs(fft(list(jaw_open_buffer)))[:buffer_size//2])
if len(jaw_open_buffer) == buffer_size else 0.0

    surface = compute_surface_vectors_split(landmarks, prev_landmarks,
left_jaw_idx_surface, right_jaw_idx_surface)
    l, r = surface['left'], surface['right']
    jaw_surface_var_buffer.append({'left': l['var'], 'right': r['var']})
    jaw_surface_dir_buffer.append({'left': l['angle'], 'right':
r['angle']})

    lv = [x['left'] for x in list(jaw_surface_var_buffer)[-10:]]
    rv = [x['right'] for x in list(jaw_surface_var_buffer)[-10:]]
    la = [x['left'] for x in list(jaw_surface_dir_buffer)[-10:]]

```



```

ra = [x['right'] for x in list(jaw_surface_dir_buffer)[-10:]]

vl = [np.mean(lv), np.std(lv), np.min(lv), np.max(lv)] if lv else [0]*4
vr = [np.mean(rv), np.std(rv), np.min(rv), np.max(rv)] if rv else [0]*4
dl = [np.mean(la), np.std(la)] if len(la) > 1 else [0, 0]
dr = [np.mean(ra), np.std(ra)] if len(ra) > 1 else [0, 0]

return {
    'Jaw opening (mean)': jaw_open_stats[0],
    'Jaw opening (std)': jaw_open_stats[1],
    'Jaw opening (min)': jaw_open_stats[2],
    'Jaw opening (max)': jaw_open_stats[3],
    'Jaw velocity (mean)': jaw_vel_stats[0],
    'Jaw velocity (std)': jaw_vel_stats[1],
    'Jaw asymmetry (mean)': jaw_asym_stats[0],
    'Jaw asymmetry (std)': jaw_asym_stats[1],
    'Jaw asymmetry (max)': jaw_asym_stats[2],
    'Jaw rapid changes count': rapid_count,
    'Jaw significant movements count': sig_mov_count,
    'Jaw frequency mean': freq_mean,
    'Jaw peak frequency': peak_freq,

    'Left surface vector magnitude mean': l['mean_mag'],
    'Left surface variance (current)': l['var'],
    'Left surface variance mean': vl[0],
    'Left surface variance std': vl[1],
    'Left surface variance min': vl[2],
    'Left surface variance max': vl[3],
    'Left surface dominant angle mean': dl[0],
    'Left surface dominant angle std': dl[1],

    'Right surface vector magnitude mean': r['mean_mag'],
    'Right surface variance (current)': r['var'],
    'Right surface variance mean': vr[0],
    'Right surface variance std': vr[1],
    'Right surface variance min': vr[2],
    'Right surface variance max': vr[3],
    'Right surface dominant angle mean': dr[0],
    'Right surface dominant angle std': dr[1],
}

# --- LIPS ---
def compute_lips_features(landmarks, prev_landmarks):
    if landmarks is None: return {}
    nose_tip = np.array(landmarks[1])
    norm_landmarks = [np.array(lm) - nose_tip for lm in landmarks]

    upper_lip = norm_landmarks[13]
    lower_lip = norm_landmarks[14]
    lip_open = np.linalg.norm(upper_lip - lower_lip)
    lips_open_buffer.append(lip_open)
    lip_open_stats = [np.mean(lips_open_buffer), np.std(lips_open_buffer),
np.min(lips_open_buffer), np.max(lips_open_buffer)] if

```

```

len(lips_open_buffer) > 1 else [0.0, 0.0, 0.0, 0.0]

    lip_vel = abs(lip_open - lips_open_buffer[-2]) if len(lips_open_buffer)
> 1 else 0
    lips_vel_buffer.append(lip_vel)
    lip_vel_stats = [np.mean(lips_vel_buffer), np.std(lips_vel_buffer)] if
len(lips_vel_buffer) > 1 else [0.0, 0.0]

    micro_var = np.var(lips_open_buffer) if len(lips_open_buffer) > 1 else
0.0
    rapid_count = len(find_peaks(list(lips_vel_buffer), distance=2)[0]) if
len(lips_vel_buffer) > 1 else 0
    sig_mov_count = sum(1 for v in lips_vel_buffer if v > 0.001)

    freq_mean = np.mean(np.abs(fft(list(lips_open_buffer)))
[:buffer_size//2]) if len(lips_open_buffer) == buffer_size else 0.0
    peak_freq = np.max(np.abs(fft(list(lips_open_buffer)))
[:buffer_size//2]) if len(lips_open_buffer) == buffer_size else 0.0

    left_corner_y = norm_landmarks[61][1] - norm_landmarks[17][1]
    right_corner_y = norm_landmarks[291][1] - norm_landmarks[17][1]
    corner_asym = np.abs(left_corner_y - right_corner_y)
    corner_asym_stats = [np.mean([corner_asym]), np.std([corner_asym]),
np.max([corner_asym])] if len(lips_open_buffer) > 1 else [0.0, 0.0, 0.0]

    surface = compute_surface_vectors_split(landmarks, prev_landmarks,
left_lip_idx_surface, right_lip_idx_surface)
    l, r = surface['left'], surface['right']
    lips_surface_var_buffer.append({'left': l['var'], 'right': r['var']})
    lips_surface_dir_buffer.append({'left': l['angle'], 'right':
r['angle']})

    lv = [x['left'] for x in list(lips_surface_var_buffer)[-10:]]
    rv = [x['right'] for x in list(lips_surface_var_buffer)[-10:]]
    la = [x['left'] for x in list(lips_surface_dir_buffer)[-10:]]
    ra = [x['right'] for x in list(lips_surface_dir_buffer)[-10:]]

    vl = [np.mean(lv), np.std(lv), np.min(lv), np.max(lv)] if lv else [0]*4
    vr = [np.mean(rv), np.std(rv), np.min(rv), np.max(rv)] if rv else [0]*4
    dl = [np.mean(la), np.std(la)] if len(la) > 1 else [0, 0]
    dr = [np.mean(ra), np.std(ra)] if len(ra) > 1 else [0, 0]

    return {
        'Lip micro-expression variance mean': micro_var,
        'Lip micro-expression rapid changes count': rapid_count,
        'Lip opening (mean)': lip_open_stats[0],
        'Lip opening (std)': lip_open_stats[1],
        'Lip opening (min)': lip_open_stats[2],
        'Lip opening (max)': lip_open_stats[3],
        'Lip velocity (mean)': lip_vel_stats[0],
        'Lip velocity (std)': lip_vel_stats[1],
        'Lip significant movements count': sig_mov_count,
        'Lip frequency mean': freq_mean,

```

```

'Lip peak frequency': peak_freq,
'Lip corner asymmetry (mean)': corner_asym_stats[0],
'Lip corner asymmetry (std)': corner_asym_stats[1],
'Lip corner asymmetry (max)': corner_asym_stats[2],

'Left surface vector magnitude mean': l['mean_mag'],
'Left surface variance (current)': l['var'],
'Left surface variance mean': vl[0],
'Left surface variance std': vl[1],
'Left surface variance min': vl[2],
'Left surface variance max': vl[3],
'Left surface dominant angle mean': dl[0],
'Left surface dominant angle std': dl[1],

'Right surface vector magnitude mean': r['mean_mag'],
'Right surface variance (current)': r['var'],
'Right surface variance mean': vr[0],
'Right surface variance std': vr[1],
'Right surface variance min': vr[2],
'Right surface variance max': vr[3],
'Right surface dominant angle mean': dr[0],
'Right surface dominant angle std': dr[1],
}

# --- MOUTH ---
def compute_mouth_features(landmarks, prev_landmarks):
    if landmarks is None: return {}
    nose_tip = np.array(landmarks[1])
    norm_landmarks = [np.array(lm) - nose_tip for lm in landmarks]

    upper_lip = norm_landmarks[13]
    lower_lip = norm_landmarks[14]
    mouth_open = np.linalg.norm(upper_lip - lower_lip)
    mouth_open_buffer.append(mouth_open)
    mouth_open_stats = [np.mean(mouth_open_buffer),
np.std(mouth_open_buffer), np.min(mouth_open_buffer),
np.max(mouth_open_buffer)] if len(mouth_open_buffer) > 1 else [0.0, 0.0,
0.0, 0.0]

    mouth_vel = abs(mouth_open - mouth_open_buffer[-2]) if
len(mouth_open_buffer) > 1 else 0
    mouth_vel_buffer.append(mouth_vel)
    mouth_vel_stats = [np.mean(mouth_vel_buffer), np.std(mouth_vel_buffer)]
if len(mouth_vel_buffer) > 1 else [0.0, 0.0]

    micro_var = np.var(mouth_open_buffer) if len(mouth_open_buffer) > 1
else 0.0
    rapid_count = len(find_peaks(list(mouth_vel_buffer), distance=2)[0]) if
len(mouth_vel_buffer) > 1 else 0
    sig_mov_count = sum(1 for v in mouth_vel_buffer if v > 0.001)

    freq_mean = np.mean(np.abs(fft(list(mouth_open_buffer))))
[:buffer_size//2]) if len(mouth_open_buffer) == buffer_size else 0.0

```

```

peak_freq = np.max(np.abs(fft(list(mouth_open_buffer))))
[:buffer_size//2]) if len(mouth_open_buffer) == buffer_size else 0.0

left_corner_y = norm_landmarks[61][1] - norm_landmarks[17][1]
right_corner_y = norm_landmarks[291][1] - norm_landmarks[17][1]
corner_asym = np.abs(left_corner_y - right_corner_y)
corner_asym_stats = [np.mean([corner_asym]), np.std([corner_asym]),
np.max([corner_asym])] if len(mouth_open_buffer) > 1 else [0.0, 0.0, 0.0]

surface = compute_surface_vectors_split(landmarks, prev_landmarks,
left_mouth_idx_surface, right_mouth_idx_surface)
l, r = surface['left'], surface['right']
mouth_surface_var_buffer.append({'left': l['var'], 'right': r['var']})
mouth_surface_dir_buffer.append({'left': l['angle'], 'right':
r['angle']})

lv = [x['left'] for x in list(mouth_surface_var_buffer)[-10:]]
rv = [x['right'] for x in list(mouth_surface_var_buffer)[-10:]]
la = [x['left'] for x in list(mouth_surface_dir_buffer)[-10:]]
ra = [x['right'] for x in list(mouth_surface_dir_buffer)[-10:]]

vl = [np.mean(lv), np.std(lv), np.min(lv), np.max(lv)] if lv else [0]*4
vr = [np.mean(rv), np.std(rv), np.min(rv), np.max(rv)] if rv else [0]*4
dl = [np.mean(la), np.std(la)] if len(la) > 1 else [0, 0]
dr = [np.mean(ra), np.std(ra)] if len(ra) > 1 else [0, 0]

return {
    'Mouth micro-expression variance mean': micro_var,
    'Mouth micro-expression rapid changes count': rapid_count,
    'Mouth opening (mean)': mouth_open_stats[0],
    'Mouth opening (std)': mouth_open_stats[1],
    'Mouth opening (min)': mouth_open_stats[2],
    'Mouth opening (max)': mouth_open_stats[3],
    'Mouth velocity (mean)': mouth_vel_stats[0],
    'Mouth velocity (std)': mouth_vel_stats[1],
    'Mouth significant movements count': sig_mov_count,
    'Mouth frequency mean': freq_mean,
    'Mouth peak frequency': peak_freq,
    'Mouth corner asymmetry (mean)': corner_asym_stats[0],
    'Mouth corner asymmetry (std)': corner_asym_stats[1],
    'Mouth corner asymmetry (max)': corner_asym_stats[2],

    'Left surface vector magnitude mean': l['mean_mag'],
    'Left surface variance (current)': l['var'],
    'Left surface variance mean': vl[0],
    'Left surface variance std': vl[1],
    'Left surface variance min': vl[2],
    'Left surface variance max': vl[3],
    'Left surface dominant angle mean': dl[0],
    'Left surface dominant angle std': dl[1],

    'Right surface vector magnitude mean': r['mean_mag'],
    'Right surface variance (current)': r['var'],

```

```
'Right surface variance mean': vr[0],  
'Right surface variance std': vr[1],  
'Right surface variance min': vr[2],  
'Right surface variance max': vr[3],  
'Right surface dominant angle mean': dr[0],  
'Right surface dominant angle std': dr[1],  
}
```

```

In [ ]: # --- Video Processing ---
def process_video():
    global frame_global, features_global, prev_landmarks_global
    cap = cv2.VideoCapture(0)
    prev_landmarks = None

    region_configs = [
        ('Brow', left_brow_idx_surface, right_brow_idx_surface, (0, 255,
0)),
        ('Cheek', left_cheek_idx_surface, right_cheek_idx_surface, (255, 0,
0)),
        ('Eye', left_eye_idx_surface, right_eye_idx_surface, (0, 0, 255)),
        ('Jaw', left_jaw_idx_surface, right_jaw_idx_surface, (255, 255,
0)),
        ('Lips', left_lip_idx_surface, right_lip_idx_surface, (255, 0,
255)),
        ('Mouth', left_mouth_idx_surface, right_mouth_idx_surface, (0, 255,
255)),
    ]

    while True:
        ret, frame = cap.read()
        if not ret: break

        rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        results = face_mesh.process(rgb)
        landmarks = None

        if results.multi_face_landmarks:
            lmks = results.multi_face_landmarks[0].landmark
            landmarks = [[lm.x, lm.y, lm.z] for lm in lmks]
            h, w, _ = frame.shape
            for idx in set(brow_landmarks_idx + cheek_landmarks_idx +
eye_landmarks_idx + jaw_landmarks_idx + lip_landmarks_idx +
mouth_landmarks_idx):
                if idx < len(lmks):
                    lm = lmks[idx]
                    cv2.circle(frame, (int(lm.x * w), int(lm.y * h)), 2,
(0, 255, 0), -1)

            # Draw arrows
            for name, l_idx, r_idx, color in region_configs:
                data = compute_surface_vectors_split(landmarks,
prev_landmarks, l_idx, r_idx)
                for side_key, side_data in [('left', data['left']),
('right', data['right'])]:
                    positions = side_data['positions']
                    mean_mag = side_data['mean_mag']
                    angle = side_data['angle']

                    if len(positions) == 0 or mean_mag < 0.0008:
                        continue

```

```

# Compute average direction vector from angle
avg_dir = np.array([np.cos(angle), np.sin(angle)])

# Compute center position
center_pos = np.mean(positions, axis=0) if positions
else np.array([0, 0, 0])
center_x = int(center_pos[0] * w)
center_y = int(center_pos[1] * h)

# Scale arrow length
base_scale = 20
max_length = 25
min_length = 3
arrow_length = min(max(mean_mag * w * base_scale * 30,
min_length), max_length)

dx = avg_dir[0] * arrow_length
dy = avg_dir[1] * arrow_length

end_x = int(center_x + dx)
end_y = int(center_y + dy)

# Draw the arrow
cv2.arrowedLine(frame, (center_x, center_y), (end_x,
end_y), color, 1, tipLength=0.25)

# Label
label = name[0] + ('L' if side_key == 'left' else 'R')
cv2.putText(frame, label, (center_x - 20, center_y -
10), cv2.FONT_HERSHEY_SIMPLEX, 0.5, color, 1)

# Compute features
features_global.update({
    'Brow': compute_brow_features(landmarks, prev_landmarks),
    'Cheek': compute_cheek_features(landmarks, prev_landmarks),
    'Eye': compute_eye_features(landmarks, prev_landmarks),
    'Jaw': compute_jaw_features(landmarks, prev_landmarks),
    'Lips': compute_lips_features(landmarks, prev_landmarks),
    'Mouth': compute_mouth_features(landmarks, prev_landmarks),
})

prev_landmarks = landmarks
frame_global = frame.copy()
time.sleep(0.03)

cap.release()

```

```
In [53]: # --- Reset Buffers ---
def reset_buffers():
    global brow_raise_buffer, brow_left_raise_buffer,
    brow_right_raise_buffer, brow_inner_raise_buffer, brow_vel_buffer,
    brow_surface_var_buffer, brow_surface_dir_buffer
    global cheek_raise_buffer, cheek_vel_buffer, cheek_surface_var_buffer,
    cheek_surface_dir_buffer
    global eye_ratio_buffer, eye_vel_buffer, blink_buffer,
    eye_surface_var_buffer, eye_surface_dir_buffer
    global jaw_open_buffer, jaw_vel_buffer, jaw_surface_var_buffer,
    jaw_surface_dir_buffer
    global lips_open_buffer, lips_vel_buffer, lips_surface_var_buffer,
    lips_surface_dir_buffer
    global mouth_open_buffer, mouth_vel_buffer, mouth_surface_var_buffer,
    mouth_surface_dir_buffer

    buffer_size = 10
    brow_raise_buffer = deque(maxlen=buffer_size)
    brow_left_raise_buffer = deque(maxlen=buffer_size)
    brow_right_raise_buffer = deque(maxlen=buffer_size)
    brow_inner_raise_buffer = deque(maxlen=buffer_size)
    brow_vel_buffer = deque(maxlen=buffer_size)
    brow_surface_var_buffer = deque(maxlen=buffer_size)
    brow_surface_dir_buffer = deque(maxlen=buffer_size)

    cheek_raise_buffer = deque(maxlen=buffer_size)
    cheek_vel_buffer = deque(maxlen=buffer_size)
    cheek_surface_var_buffer = deque(maxlen=buffer_size)
    cheek_surface_dir_buffer = deque(maxlen=buffer_size)

    eye_ratio_buffer = deque(maxlen=buffer_size)
    eye_vel_buffer = deque(maxlen=buffer_size)
    blink_buffer = deque(maxlen=30)
    eye_surface_var_buffer = deque(maxlen=buffer_size)
    eye_surface_dir_buffer = deque(maxlen=buffer_size)

    jaw_open_buffer = deque(maxlen=buffer_size)
    jaw_vel_buffer = deque(maxlen=buffer_size)
    jaw_surface_var_buffer = deque(maxlen=buffer_size)
    jaw_surface_dir_buffer = deque(maxlen=buffer_size)

    lips_open_buffer = deque(maxlen=buffer_size)
    lips_vel_buffer = deque(maxlen=buffer_size)
    lips_surface_var_buffer = deque(maxlen=buffer_size)
    lips_surface_dir_buffer = deque(maxlen=buffer_size)

    mouth_open_buffer = deque(maxlen=buffer_size)
    mouth_vel_buffer = deque(maxlen=buffer_size)
    mouth_surface_var_buffer = deque(maxlen=buffer_size)
    mouth_surface_dir_buffer = deque(maxlen=buffer_size)
```



```

In [54]: def extract_features_from_video(video_path, apply_augmentation=False):
    if not os.path.exists(video_path):
        print(f"Error: Video file not found at {video_path}")
        return []

    # Re-initialize FaceMesh with lower thresholds for better detection
    mp_face_mesh = mp.solutions.face_mesh
    face_mesh = mp_face_mesh.FaceMesh(
        static_image_mode=False,
        max_num_faces=1,
        refine_landmarks=True,
        min_detection_confidence=0.1,
        min_tracking_confidence=0.1
    )

    cap = cv2.VideoCapture(video_path)
    if not cap.isOpened():
        print(f"Error: Cannot open video at {video_path}")
        return []

    features_list = []
    frame_count = 0
    detected_count = 0
    prev_landmarks = None

    while cap.isOpened():
        ret, frame = cap.read()
        if not ret:
            break
        frame_count += 1

        # Process frame
        rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        results = face_mesh.process(rgb_frame)

        if results.multi_face_landmarks:
            detected_count += 1
            landmarks = [(lm.x, lm.y, lm.z) for lm in
results.multi_face_landmarks[0].landmark]

            # Apply rotation/distance normalization
            norm_landmarks = normalize_for_rotation_distance(landmarks,
prev_landmarks)

            # Compute features
            brow_features = compute_brow_features(norm_landmarks,
prev_landmarks)
            cheek_features = compute_cheek_features(norm_landmarks,
prev_landmarks)
            eye_features = compute_eye_features(norm_landmarks,
prev_landmarks)
            jaw_features = compute_jaw_features(norm_landmarks,
prev_landmarks)

```

```

        lips_features = compute_lips_features(norm_landmarks,
prev_landmarks)
        mouth_features = compute_mouth_features(norm_landmarks,
prev_landmarks)

        features = {
            'Brow': brow_features,
            'Cheek': cheek_features,
            'Eye': eye_features,
            'Jaw': jaw_features,
            'Lips': lips_features,
            'Mouth': mouth_features
        }
        features_list.append(features)
        prev_landmarks = landmarks

    cap.release()
    face_mesh.close()

    print(f"Processed {frame_count} frames.")
    print(f"Detected faces in {detected_count} frames ({(detected_count /
frame_count * 100 if frame_count > 0 else 0):.2f}%).")

    if detected_count == 0:
        print("Tip: If no detections, try a different video, check
lighting/angle, or further lower confidence thresholds.")

    return features_list

```

This function takes the nested dictionary of facial features (produced by your MediaPipe-based feature extraction) and converts it into a single flat 1D NumPy array (a vector of numbers). This flat vector is what your PyTorch CNN+LSTM model needs as input for each frame.

```

In [55]: # --- Flatten Features ---
def flatten_features(features):
    flat = []
    for region in ['Brow', 'Cheek', 'Eye', 'Jaw', 'Lips', 'Mouth']:
        if region in features:
            for v in features[region].values():
                flat.append(float(v)) # Ensure float
    return np.array(flat)

```

This section of responsible for loading all the videos from your dataset folder, extracting facial features from each video using MediaPipe, converting them into sequences of flat vectors, and building the training data (X and y) for your PyTorch model.

```
In [57]: # --- Load Dataset ---
data_dir = 'expressions'
labels = ['Angry', 'Calm', 'Disgust', 'Fearful', 'Happy', 'Neutral', 'Sad', 'Surprised']
X = []
y = []
lengths = []

for label_idx, label in enumerate(labels):
    subdir = os.path.join(data_dir, label)
    if os.path.isdir(subdir):
        for vid in os.listdir(subdir):
            if vid.lower().endswith(('.mp4', '.avi', '.mov')):
                path = os.path.join(subdir, vid)
                features_list = extract_features_from_video(path)
                if features_list:
                    seq = [flatten_features(f) for f in features_list if f]
                    if len(seq) > 0:
                        X.append(seq)
                        y.append(label_idx)
                        lengths.append(len(seq)) # Store length

if len(X) == 0:
    print("No videos found or processed.")
    exit()
```

```
Processed 115 frames.
Detected faces in 115 frames (100.00%).
Processed 110 frames.
Detected faces in 110 frames (100.00%).
Processed 120 frames.
Detected faces in 120 frames (100.00%).
Processed 111 frames.
Detected faces in 111 frames (100.00%).
Processed 116 frames.
Detected faces in 116 frames (100.00%).
Processed 112 frames.
Detected faces in 112 frames (100.00%).
Processed 120 frames.
Detected faces in 120 frames (100.00%).
Processed 111 frames.
Detected faces in 111 frames (100.00%).
Processed 106 frames.
```

Detected faces in 106 frames (100.00%).
Processed 117 frames.
Detected faces in 117 frames (100.00%).
Processed 108 frames.
Detected faces in 108 frames (100.00%).
Processed 127 frames.
Detected faces in 127 frames (100.00%).
Processed 103 frames.
Detected faces in 103 frames (100.00%).
Processed 107 frames.
Detected faces in 107 frames (100.00%).
Processed 119 frames.
Detected faces in 119 frames (100.00%).
Processed 103 frames.
Detected faces in 103 frames (100.00%).
Processed 117 frames.
Detected faces in 117 frames (100.00%).
Processed 112 frames.
Detected faces in 112 frames (100.00%).
Processed 97 frames.
Detected faces in 97 frames (100.00%).
Processed 124 frames.
Detected faces in 124 frames (100.00%).
Processed 111 frames.
Detected faces in 111 frames (100.00%).
Processed 99 frames.
Detected faces in 99 frames (100.00%).
Processed 123 frames.
Detected faces in 123 frames (100.00%).
Processed 117 frames.
Detected faces in 117 frames (100.00%).
Processed 110 frames.
Detected faces in 110 frames (100.00%).
Processed 122 frames.
Detected faces in 122 frames (100.00%).
Processed 108 frames.
Detected faces in 108 frames (100.00%).
Processed 114 frames.
Detected faces in 114 frames (100.00%).
Processed 118 frames.
Detected faces in 118 frames (100.00%).
Processed 119 frames.
Detected faces in 119 frames (100.00%).
Processed 108 frames.
Detected faces in 108 frames (100.00%).
Processed 106 frames.
Detected faces in 106 frames (100.00%).
Processed 120 frames.
Detected faces in 120 frames (100.00%).
Processed 104 frames.
Detected faces in 104 frames (100.00%).
Processed 118 frames.
Detected faces in 118 frames (100.00%).

Processed 106 frames.
Detected faces in 106 frames (100.00%).
Processed 110 frames.
Detected faces in 110 frames (100.00%).
Processed 107 frames.
Detected faces in 107 frames (100.00%).
Processed 120 frames.
Detected faces in 120 frames (100.00%).
Processed 110 frames.
Detected faces in 110 frames (100.00%).
Processed 125 frames.
Detected faces in 125 frames (100.00%).
Processed 110 frames.
Detected faces in 110 frames (100.00%).
Processed 112 frames.
Detected faces in 112 frames (100.00%).
Processed 112 frames.
Detected faces in 112 frames (100.00%).
Processed 99 frames.
Detected faces in 99 frames (100.00%).
Processed 112 frames.
Detected faces in 112 frames (100.00%).
Processed 122 frames.
Detected faces in 122 frames (100.00%).
Processed 117 frames.
Detected faces in 117 frames (100.00%).
Processed 132 frames.
Detected faces in 132 frames (100.00%).
Processed 113 frames.
Detected faces in 113 frames (100.00%).
Processed 102 frames.
Detected faces in 102 frames (100.00%).
Processed 122 frames.
Detected faces in 122 frames (100.00%).
Processed 108 frames.
Detected faces in 108 frames (100.00%).
Processed 119 frames.
Detected faces in 119 frames (100.00%).
Processed 113 frames.
Detected faces in 113 frames (100.00%).
Processed 125 frames.
Detected faces in 125 frames (100.00%).
Processed 106 frames.
Detected faces in 106 frames (100.00%).
Processed 108 frames.
Detected faces in 108 frames (100.00%).
Processed 110 frames.
Detected faces in 110 frames (100.00%).
Processed 127 frames.
Detected faces in 127 frames (100.00%).
Processed 105 frames.
Detected faces in 105 frames (100.00%).
Processed 119 frames.

Detected faces in 119 frames (100.00%).
Processed 104 frames.
Detected faces in 104 frames (100.00%).
Processed 111 frames.
Detected faces in 111 frames (100.00%).
Processed 118 frames.
Detected faces in 118 frames (100.00%).
Processed 124 frames.
Detected faces in 124 frames (100.00%).
Processed 131 frames.
Detected faces in 131 frames (100.00%).
Processed 116 frames.
Detected faces in 116 frames (100.00%).
Processed 102 frames.
Detected faces in 102 frames (100.00%).
Processed 131 frames.
Detected faces in 131 frames (100.00%).
Processed 130 frames.
Detected faces in 130 frames (100.00%).
Processed 120 frames.
Detected faces in 120 frames (100.00%).
Processed 132 frames.
Detected faces in 132 frames (100.00%).
Processed 116 frames.
Detected faces in 116 frames (100.00%).
Processed 107 frames.
Detected faces in 107 frames (100.00%).
Processed 125 frames.
Detected faces in 125 frames (100.00%).
Processed 120 frames.
Detected faces in 120 frames (100.00%).
Processed 113 frames.
Detected faces in 113 frames (100.00%).
Processed 112 frames.
Detected faces in 112 frames (100.00%).
Processed 134 frames.
Detected faces in 134 frames (100.00%).
Processed 119 frames.
Detected faces in 119 frames (100.00%).
Processed 109 frames.
Detected faces in 109 frames (100.00%).
Processed 128 frames.
Detected faces in 128 frames (100.00%).
Processed 130 frames.
Detected faces in 130 frames (100.00%).
Processed 111 frames.
Detected faces in 111 frames (100.00%).
Processed 112 frames.
Detected faces in 112 frames (100.00%).
Processed 111 frames.
Detected faces in 111 frames (100.00%).
Processed 113 frames.
Detected faces in 113 frames (100.00%).

Processed 143 frames.
Detected faces in 143 frames (100.00%).
Processed 122 frames.
Detected faces in 122 frames (100.00%).
Processed 134 frames.
Detected faces in 134 frames (100.00%).
Processed 117 frames.
Detected faces in 117 frames (100.00%).
Processed 100 frames.
Detected faces in 100 frames (100.00%).
Processed 131 frames.
Detected faces in 131 frames (100.00%).
Processed 105 frames.
Detected faces in 105 frames (100.00%).
Processed 115 frames.
Detected faces in 115 frames (100.00%).
Processed 115 frames.
Detected faces in 115 frames (100.00%).
Processed 114 frames.
Detected faces in 114 frames (100.00%).
Processed 121 frames.
Detected faces in 121 frames (100.00%).
Processed 120 frames.
Detected faces in 120 frames (100.00%).
Processed 104 frames.
Detected faces in 104 frames (100.00%).
Processed 108 frames.
Detected faces in 108 frames (100.00%).
Processed 99 frames.
Detected faces in 99 frames (100.00%).
Processed 107 frames.
Detected faces in 107 frames (100.00%).
Processed 98 frames.
Detected faces in 98 frames (100.00%).
Processed 115 frames.
Detected faces in 115 frames (100.00%).
Processed 99 frames.
Detected faces in 99 frames (100.00%).
Processed 106 frames.
Detected faces in 106 frames (100.00%).
Processed 111 frames.
Detected faces in 111 frames (100.00%).
Processed 104 frames.
Detected faces in 104 frames (100.00%).
Processed 104 frames.
Detected faces in 104 frames (100.00%).
Processed 109 frames.
Detected faces in 109 frames (100.00%).
Processed 109 frames.
Detected faces in 109 frames (100.00%).
Processed 121 frames.
Detected faces in 121 frames (100.00%).
Processed 106 frames.

Detected faces in 106 frames (100.00%).
Processed 119 frames.
Detected faces in 119 frames (100.00%).
Processed 107 frames.
Detected faces in 107 frames (100.00%).
Processed 107 frames.
Detected faces in 107 frames (100.00%).
Processed 120 frames.
Detected faces in 120 frames (100.00%).
Processed 112 frames.
Detected faces in 112 frames (100.00%).
Processed 112 frames.
Detected faces in 112 frames (100.00%).
Processed 123 frames.
Detected faces in 123 frames (100.00%).
Processed 122 frames.
Detected faces in 122 frames (100.00%).
Processed 112 frames.
Detected faces in 112 frames (100.00%).
Processed 102 frames.
Detected faces in 102 frames (100.00%).
Processed 108 frames.
Detected faces in 108 frames (100.00%).
Processed 100 frames.
Detected faces in 100 frames (100.00%).
Processed 112 frames.
Detected faces in 112 frames (100.00%).
Processed 95 frames.
Detected faces in 95 frames (100.00%).
Processed 106 frames.
Detected faces in 106 frames (100.00%).
Processed 106 frames.
Detected faces in 106 frames (100.00%).
Processed 116 frames.
Detected faces in 116 frames (100.00%).
Processed 105 frames.
Detected faces in 105 frames (100.00%).
Processed 102 frames.
Detected faces in 102 frames (100.00%).
Processed 109 frames.
Detected faces in 109 frames (100.00%).
Processed 107 frames.
Detected faces in 107 frames (100.00%).
Processed 120 frames.
Detected faces in 120 frames (100.00%).
Processed 106 frames.
Detected faces in 106 frames (100.00%).
Processed 122 frames.
Detected faces in 122 frames (100.00%).
Processed 105 frames.
Detected faces in 105 frames (100.00%).
Processed 110 frames.
Detected faces in 110 frames (100.00%).

Processed 124 frames.
Detected faces in 124 frames (100.00%).
Processed 121 frames.
Detected faces in 121 frames (100.00%).
Processed 119 frames.
Detected faces in 119 frames (100.00%).
Processed 142 frames.
Detected faces in 142 frames (100.00%).
Processed 134 frames.
Detected faces in 134 frames (100.00%).
Processed 120 frames.
Detected faces in 120 frames (100.00%).
Processed 109 frames.
Detected faces in 109 frames (100.00%).
Processed 99 frames.
Detected faces in 99 frames (100.00%).
Processed 112 frames.
Detected faces in 112 frames (100.00%).
Processed 115 frames.
Detected faces in 115 frames (100.00%).
Processed 118 frames.
Detected faces in 118 frames (100.00%).
Processed 94 frames.
Detected faces in 94 frames (100.00%).
Processed 115 frames.
Detected faces in 115 frames (100.00%).
Processed 109 frames.
Detected faces in 109 frames (100.00%).
Processed 116 frames.
Detected faces in 116 frames (100.00%).
Processed 115 frames.
Detected faces in 115 frames (100.00%).
Processed 108 frames.
Detected faces in 108 frames (100.00%).
Processed 133 frames.
Detected faces in 133 frames (100.00%).
Processed 116 frames.
Detected faces in 116 frames (100.00%).
Processed 120 frames.
Detected faces in 120 frames (100.00%).
Processed 111 frames.
Detected faces in 111 frames (100.00%).
Processed 138 frames.
Detected faces in 138 frames (100.00%).
Processed 123 frames.
Detected faces in 123 frames (100.00%).
Processed 119 frames.
Detected faces in 119 frames (100.00%).
Processed 121 frames.
Detected faces in 121 frames (100.00%).
Processed 119 frames.
Detected faces in 119 frames (100.00%).
Processed 114 frames.

Detected faces in 114 frames (100.00%).
Processed 135 frames.
Detected faces in 135 frames (100.00%).
Processed 138 frames.
Detected faces in 138 frames (100.00%).
Processed 116 frames.
Detected faces in 116 frames (100.00%).
Processed 112 frames.
Detected faces in 112 frames (100.00%).
Processed 102 frames.
Detected faces in 102 frames (100.00%).
Processed 112 frames.
Detected faces in 112 frames (100.00%).
Processed 112 frames.
Detected faces in 112 frames (100.00%).
Processed 112 frames.
Detected faces in 112 frames (100.00%).
Processed 96 frames.
Detected faces in 96 frames (100.00%).
Processed 116 frames.
Detected faces in 116 frames (100.00%).
Processed 113 frames.
Detected faces in 113 frames (100.00%).
Processed 118 frames.
Detected faces in 118 frames (100.00%).
Processed 107 frames.
Detected faces in 107 frames (100.00%).
Processed 108 frames.
Detected faces in 108 frames (100.00%).
Processed 120 frames.
Detected faces in 120 frames (100.00%).
Processed 122 frames.
Detected faces in 122 frames (100.00%).
Processed 125 frames.
Detected faces in 125 frames (100.00%).
Processed 115 frames.
Detected faces in 115 frames (100.00%).
Processed 138 frames.
Detected faces in 138 frames (100.00%).
Processed 122 frames.
Detected faces in 122 frames (100.00%).
Processed 115 frames.
Detected faces in 115 frames (100.00%).
Processed 110 frames.
Detected faces in 110 frames (100.00%).
Processed 121 frames.
Detected faces in 121 frames (100.00%).
Processed 117 frames.
Detected faces in 117 frames (100.00%).
Processed 130 frames.
Detected faces in 130 frames (100.00%).
Processed 104 frames.
Detected faces in 104 frames (100.00%).

Processed 106 frames.
Detected faces in 106 frames (100.00%).
Processed 117 frames.
Detected faces in 117 frames (100.00%).
Processed 101 frames.
Detected faces in 101 frames (100.00%).
Processed 111 frames.
Detected faces in 111 frames (100.00%).
Processed 111 frames.
Detected faces in 111 frames (100.00%).
Processed 94 frames.
Detected faces in 94 frames (100.00%).
Processed 112 frames.
Detected faces in 112 frames (100.00%).
Processed 111 frames.
Detected faces in 111 frames (100.00%).
Processed 113 frames.
Detected faces in 113 frames (100.00%).
Processed 114 frames.
Detected faces in 114 frames (100.00%).
Processed 129 frames.
Detected faces in 129 frames (100.00%).
Processed 128 frames.
Detected faces in 128 frames (100.00%).
Processed 116 frames.
Detected faces in 116 frames (100.00%).
Processed 114 frames.
Detected faces in 114 frames (100.00%).
Processed 111 frames.
Detected faces in 111 frames (100.00%).
Processed 107 frames.
Detected faces in 107 frames (100.00%).
Processed 129 frames.
Detected faces in 129 frames (100.00%).
Processed 116 frames.
Detected faces in 116 frames (100.00%).
Processed 109 frames.
Detected faces in 109 frames (100.00%).
Processed 119 frames.
Detected faces in 119 frames (100.00%).
Processed 117 frames.
Detected faces in 117 frames (100.00%).
Processed 123 frames.
Detected faces in 123 frames (100.00%).
Processed 127 frames.
Detected faces in 127 frames (100.00%).
Processed 105 frames.
Detected faces in 105 frames (100.00%).
Processed 110 frames.
Detected faces in 110 frames (100.00%).
Processed 104 frames.
Detected faces in 104 frames (100.00%).
Processed 126 frames.

Detected faces in 126 frames (100.00%).
Processed 108 frames.
Detected faces in 108 frames (100.00%).
Processed 110 frames.
Detected faces in 110 frames (100.00%).
Processed 95 frames.
Detected faces in 95 frames (100.00%).
Processed 108 frames.
Detected faces in 108 frames (100.00%).
Processed 108 frames.
Detected faces in 108 frames (100.00%).
Processed 112 frames.
Detected faces in 112 frames (100.00%).
Processed 110 frames.
Detected faces in 110 frames (100.00%).
Processed 122 frames.
Detected faces in 122 frames (100.00%).
Processed 144 frames.
Detected faces in 144 frames (100.00%).
Processed 112 frames.
Detected faces in 112 frames (100.00%).
Processed 122 frames.
Detected faces in 122 frames (100.00%).
Processed 117 frames.
Detected faces in 117 frames (100.00%).
Processed 107 frames.
Detected faces in 107 frames (100.00%).
Processed 123 frames.
Detected faces in 123 frames (100.00%).
Processed 128 frames.
Detected faces in 128 frames (100.00%).
Processed 124 frames.
Detected faces in 124 frames (100.00%).
Processed 123 frames.
Detected faces in 123 frames (100.00%).
Processed 120 frames.
Detected faces in 120 frames (100.00%).
Processed 130 frames.
Detected faces in 130 frames (100.00%).
Processed 128 frames.
Detected faces in 128 frames (100.00%).
Processed 124 frames.
Detected faces in 124 frames (100.00%).
Processed 122 frames.
Detected faces in 122 frames (100.00%).
Processed 108 frames.
Detected faces in 108 frames (100.00%).
Processed 140 frames.
Detected faces in 140 frames (100.00%).
Processed 114 frames.
Detected faces in 114 frames (100.00%).
Processed 118 frames.
Detected faces in 118 frames (100.00%).

Processed 102 frames.
Detected faces in 102 frames (100.00%).
Processed 117 frames.
Detected faces in 117 frames (100.00%).
Processed 110 frames.
Detected faces in 110 frames (100.00%).
Processed 124 frames.
Detected faces in 124 frames (100.00%).
Processed 125 frames.
Detected faces in 125 frames (100.00%).
Processed 143 frames.
Detected faces in 143 frames (100.00%).
Processed 143 frames.
Detected faces in 143 frames (100.00%).
Processed 123 frames.
Detected faces in 123 frames (100.00%).
Processed 122 frames.
Detected faces in 122 frames (100.00%).
Processed 127 frames.
Detected faces in 127 frames (100.00%).
Processed 122 frames.
Detected faces in 122 frames (100.00%).
Processed 130 frames.
Detected faces in 130 frames (100.00%).
Processed 122 frames.
Detected faces in 122 frames (100.00%).
Processed 124 frames.
Detected faces in 124 frames (100.00%).
Processed 123 frames.
Detected faces in 123 frames (100.00%).
Processed 121 frames.
Detected faces in 121 frames (100.00%).
Processed 117 frames.
Detected faces in 117 frames (100.00%).
Processed 117 frames.
Detected faces in 117 frames (100.00%).
Processed 132 frames.
Detected faces in 132 frames (100.00%).
Processed 113 frames.
Detected faces in 113 frames (100.00%).
Processed 105 frames.
Detected faces in 105 frames (100.00%).
Processed 136 frames.
Detected faces in 136 frames (100.00%).
Processed 114 frames.
Detected faces in 114 frames (100.00%).
Processed 135 frames.
Detected faces in 135 frames (100.00%).
Processed 111 frames.
Detected faces in 111 frames (100.00%).
Processed 111 frames.
Detected faces in 111 frames (100.00%).
Processed 100 frames.

Detected faces in 100 frames (100.00%).
Processed 127 frames.
Detected faces in 127 frames (100.00%).
Processed 134 frames.
Detected faces in 134 frames (100.00%).
Processed 157 frames.
Detected faces in 157 frames (100.00%).
Processed 123 frames.
Detected faces in 123 frames (100.00%).
Processed 121 frames.
Detected faces in 121 frames (100.00%).
Processed 125 frames.
Detected faces in 125 frames (100.00%).
Processed 116 frames.
Detected faces in 116 frames (100.00%).
Processed 139 frames.
Detected faces in 139 frames (100.00%).
Processed 109 frames.
Detected faces in 109 frames (100.00%).
Processed 109 frames.
Detected faces in 109 frames (100.00%).
Processed 112 frames.
Detected faces in 112 frames (100.00%).
Processed 99 frames.
Detected faces in 99 frames (100.00%).
Processed 94 frames.
Detected faces in 94 frames (100.00%).
Processed 108 frames.
Detected faces in 108 frames (100.00%).
Processed 103 frames.
Detected faces in 103 frames (100.00%).
Processed 119 frames.
Detected faces in 119 frames (100.00%).
Processed 94 frames.
Detected faces in 94 frames (100.00%).
Processed 102 frames.
Detected faces in 102 frames (100.00%).
Processed 104 frames.
Detected faces in 104 frames (100.00%).
Processed 101 frames.
Detected faces in 101 frames (100.00%).
Processed 100 frames.
Detected faces in 100 frames (100.00%).
Processed 98 frames.
Detected faces in 98 frames (100.00%).
Processed 111 frames.
Detected faces in 111 frames (100.00%).
Processed 103 frames.
Detected faces in 103 frames (100.00%).
Processed 103 frames.
Detected faces in 103 frames (100.00%).
Processed 110 frames.
Detected faces in 110 frames (100.00%).

Processed 114 frames.
Detected faces in 114 frames (100.00%).
Processed 110 frames.
Detected faces in 110 frames (100.00%).
Processed 107 frames.
Detected faces in 107 frames (100.00%).
Processed 103 frames.
Detected faces in 103 frames (100.00%).
Processed 123 frames.
Detected faces in 123 frames (100.00%).
Processed 112 frames.
Detected faces in 112 frames (100.00%).
Processed 115 frames.
Detected faces in 115 frames (100.00%).
Processed 97 frames.
Detected faces in 97 frames (100.00%).
Processed 101 frames.
Detected faces in 101 frames (100.00%).
Processed 102 frames.
Detected faces in 102 frames (100.00%).
Processed 105 frames.
Detected faces in 105 frames (100.00%).
Processed 110 frames.
Detected faces in 110 frames (100.00%).
Processed 94 frames.
Detected faces in 94 frames (100.00%).
Processed 99 frames.
Detected faces in 99 frames (100.00%).
Processed 105 frames.
Detected faces in 105 frames (100.00%).
Processed 104 frames.
Detected faces in 104 frames (100.00%).
Processed 95 frames.
Detected faces in 95 frames (100.00%).
Processed 106 frames.
Detected faces in 106 frames (100.00%).
Processed 98 frames.
Detected faces in 98 frames (100.00%).
Processed 105 frames.
Detected faces in 105 frames (100.00%).
Processed 104 frames.
Detected faces in 104 frames (100.00%).
Processed 102 frames.
Detected faces in 102 frames (100.00%).
Processed 104 frames.
Detected faces in 104 frames (100.00%).
Processed 131 frames.
Detected faces in 131 frames (100.00%).
Processed 113 frames.
Detected faces in 113 frames (100.00%).
Processed 108 frames.
Detected faces in 108 frames (100.00%).
Processed 97 frames.

Detected faces in 97 frames (100.00%).
Processed 122 frames.
Detected faces in 122 frames (100.00%).
Processed 149 frames.
Detected faces in 149 frames (100.00%).
Processed 114 frames.
Detected faces in 114 frames (100.00%).
Processed 99 frames.
Detected faces in 99 frames (100.00%).
Processed 118 frames.
Detected faces in 118 frames (100.00%).
Processed 97 frames.
Detected faces in 97 frames (100.00%).
Processed 108 frames.
Detected faces in 108 frames (100.00%).
Processed 109 frames.
Detected faces in 109 frames (100.00%).
Processed 119 frames.
Detected faces in 119 frames (100.00%).
Processed 96 frames.
Detected faces in 96 frames (100.00%).
Processed 105 frames.
Detected faces in 105 frames (100.00%).
Processed 96 frames.
Detected faces in 96 frames (100.00%).
Processed 111 frames.
Detected faces in 111 frames (100.00%).
Processed 101 frames.
Detected faces in 101 frames (100.00%).
Processed 100 frames.
Detected faces in 100 frames (100.00%).
Processed 99 frames.
Detected faces in 99 frames (100.00%).
Processed 112 frames.
Detected faces in 112 frames (100.00%).
Processed 110 frames.
Detected faces in 110 frames (100.00%).
Processed 112 frames.
Detected faces in 112 frames (100.00%).
Processed 114 frames.
Detected faces in 114 frames (100.00%).
Processed 127 frames.
Detected faces in 127 frames (100.00%).
Processed 119 frames.
Detected faces in 119 frames (100.00%).
Processed 107 frames.
Detected faces in 107 frames (100.00%).
Processed 107 frames.
Detected faces in 107 frames (100.00%).
Processed 115 frames.
Detected faces in 115 frames (100.00%).
Processed 126 frames.
Detected faces in 126 frames (100.00%).

Processed 109 frames.
Detected faces in 109 frames (100.00%).
Processed 117 frames.
Detected faces in 117 frames (100.00%).
Processed 102 frames.
Detected faces in 102 frames (100.00%).
Processed 107 frames.
Detected faces in 107 frames (100.00%).
Processed 107 frames.
Detected faces in 107 frames (100.00%).
Processed 104 frames.
Detected faces in 104 frames (100.00%).
Processed 117 frames.
Detected faces in 117 frames (100.00%).
Processed 97 frames.
Detected faces in 97 frames (100.00%).
Processed 111 frames.
Detected faces in 111 frames (100.00%).
Processed 96 frames.
Detected faces in 96 frames (100.00%).
Processed 110 frames.
Detected faces in 110 frames (100.00%).
Processed 98 frames.
Detected faces in 98 frames (100.00%).
Processed 101 frames.
Detected faces in 101 frames (100.00%).
Processed 103 frames.
Detected faces in 103 frames (100.00%).
Processed 112 frames.
Detected faces in 112 frames (100.00%).
Processed 111 frames.
Detected faces in 111 frames (100.00%).
Processed 125 frames.
Detected faces in 125 frames (100.00%).
Processed 130 frames.
Detected faces in 130 frames (100.00%).
Processed 125 frames.
Detected faces in 125 frames (100.00%).
Processed 115 frames.
Detected faces in 115 frames (100.00%).
Processed 107 frames.
Detected faces in 107 frames (100.00%).
Processed 103 frames.
Detected faces in 103 frames (100.00%).
Processed 122 frames.
Detected faces in 122 frames (100.00%).
Processed 103 frames.
Detected faces in 103 frames (100.00%).
Processed 108 frames.
Detected faces in 108 frames (100.00%).
Processed 113 frames.
Detected faces in 113 frames (100.00%).
Processed 105 frames.

Detected faces in 105 frames (100.00%).
Processed 120 frames.
Detected faces in 120 frames (100.00%).
Processed 105 frames.
Detected faces in 105 frames (100.00%).
Processed 105 frames.
Detected faces in 105 frames (100.00%).
Processed 105 frames.
Detected faces in 105 frames (100.00%).
Processed 105 frames.
Detected faces in 105 frames (100.00%).
Processed 93 frames.
Detected faces in 93 frames (100.00%).
Processed 110 frames.
Detected faces in 110 frames (100.00%).
Processed 97 frames.
Detected faces in 97 frames (100.00%).
Processed 110 frames.
Detected faces in 110 frames (100.00%).
Processed 101 frames.
Detected faces in 101 frames (100.00%).
Processed 111 frames.
Detected faces in 111 frames (100.00%).
Processed 103 frames.
Detected faces in 103 frames (100.00%).
Processed 102 frames.
Detected faces in 102 frames (100.00%).
Processed 108 frames.
Detected faces in 108 frames (100.00%).
Processed 106 frames.
Detected faces in 106 frames (100.00%).
Processed 115 frames.
Detected faces in 115 frames (100.00%).
Processed 106 frames.
Detected faces in 106 frames (100.00%).
Processed 100 frames.
Detected faces in 100 frames (100.00%).
Processed 108 frames.
Detected faces in 108 frames (100.00%).
Processed 103 frames.
Detected faces in 103 frames (100.00%).
Processed 111 frames.
Detected faces in 111 frames (100.00%).
Processed 108 frames.
Detected faces in 108 frames (100.00%).
Processed 106 frames.
Detected faces in 106 frames (100.00%).
Processed 97 frames.
Detected faces in 97 frames (100.00%).
Processed 107 frames.
Detected faces in 107 frames (100.00%).
Processed 104 frames.
Detected faces in 104 frames (100.00%).

Processed 103 frames.
Detected faces in 103 frames (100.00%).
Processed 105 frames.
Detected faces in 105 frames (100.00%).
Processed 104 frames.
Detected faces in 104 frames (100.00%).
Processed 96 frames.
Detected faces in 96 frames (100.00%).
Processed 111 frames.
Detected faces in 111 frames (100.00%).
Processed 96 frames.
Detected faces in 96 frames (100.00%).
Processed 109 frames.
Detected faces in 109 frames (100.00%).
Processed 99 frames.
Detected faces in 99 frames (100.00%).
Processed 110 frames.
Detected faces in 110 frames (100.00%).
Processed 105 frames.
Detected faces in 105 frames (100.00%).
Processed 101 frames.
Detected faces in 101 frames (100.00%).
Processed 110 frames.
Detected faces in 110 frames (100.00%).
Processed 111 frames.
Detected faces in 111 frames (100.00%).
Processed 114 frames.
Detected faces in 114 frames (100.00%).
Processed 103 frames.
Detected faces in 103 frames (100.00%).
Processed 98 frames.
Detected faces in 98 frames (100.00%).
Processed 117 frames.
Detected faces in 117 frames (100.00%).
Processed 109 frames.
Detected faces in 109 frames (100.00%).
Processed 111 frames.
Detected faces in 111 frames (100.00%).
Processed 129 frames.
Detected faces in 129 frames (100.00%).
Processed 116 frames.
Detected faces in 116 frames (100.00%).
Processed 124 frames.
Detected faces in 124 frames (100.00%).
Processed 104 frames.
Detected faces in 104 frames (100.00%).
Processed 123 frames.
Detected faces in 123 frames (100.00%).
Processed 113 frames.
Detected faces in 113 frames (100.00%).
Processed 112 frames.
Detected faces in 112 frames (100.00%).
Processed 115 frames.

Detected faces in 115 frames (100.00%).
Processed 121 frames.
Detected faces in 121 frames (100.00%).
Processed 125 frames.
Detected faces in 125 frames (100.00%).
Processed 103 frames.
Detected faces in 103 frames (100.00%).
Processed 112 frames.
Detected faces in 112 frames (100.00%).
Processed 106 frames.
Detected faces in 106 frames (100.00%).
Processed 113 frames.
Detected faces in 113 frames (100.00%).
Processed 111 frames.
Detected faces in 111 frames (100.00%).
Processed 113 frames.
Detected faces in 113 frames (100.00%).
Processed 112 frames.
Detected faces in 112 frames (100.00%).
Processed 110 frames.
Detected faces in 110 frames (100.00%).
Processed 114 frames.
Detected faces in 114 frames (100.00%).
Processed 112 frames.
Detected faces in 112 frames (100.00%).
Processed 97 frames.
Detected faces in 97 frames (100.00%).
Processed 120 frames.
Detected faces in 120 frames (100.00%).
Processed 108 frames.
Detected faces in 108 frames (100.00%).
Processed 110 frames.
Detected faces in 110 frames (100.00%).
Processed 132 frames.
Detected faces in 132 frames (100.00%).
Processed 107 frames.
Detected faces in 107 frames (100.00%).
Processed 116 frames.
Detected faces in 116 frames (100.00%).
Processed 103 frames.
Detected faces in 103 frames (100.00%).
Processed 121 frames.
Detected faces in 121 frames (100.00%).
Processed 117 frames.
Detected faces in 117 frames (100.00%).
Processed 124 frames.
Detected faces in 124 frames (100.00%).
Processed 105 frames.
Detected faces in 105 frames (100.00%).
Processed 109 frames.
Detected faces in 109 frames (100.00%).
Processed 123 frames.
Detected faces in 123 frames (100.00%).

Processed 108 frames.
Detected faces in 108 frames (100.00%).
Processed 111 frames.
Detected faces in 111 frames (100.00%).
Processed 103 frames.
Detected faces in 103 frames (100.00%).
Processed 107 frames.
Detected faces in 107 frames (100.00%).
Processed 122 frames.
Detected faces in 122 frames (100.00%).
Processed 118 frames.
Detected faces in 118 frames (100.00%).
Processed 113 frames.
Detected faces in 113 frames (100.00%).
Processed 129 frames.
Detected faces in 129 frames (100.00%).
Processed 118 frames.
Detected faces in 118 frames (100.00%).
Processed 115 frames.
Detected faces in 115 frames (100.00%).
Processed 113 frames.
Detected faces in 113 frames (100.00%).
Processed 98 frames.
Detected faces in 98 frames (100.00%).
Processed 109 frames.
Detected faces in 109 frames (100.00%).
Processed 99 frames.
Detected faces in 99 frames (100.00%).
Processed 108 frames.
Detected faces in 108 frames (100.00%).
Processed 99 frames.
Detected faces in 99 frames (100.00%).
Processed 117 frames.
Detected faces in 117 frames (100.00%).
Processed 105 frames.
Detected faces in 105 frames (100.00%).
Processed 97 frames.
Detected faces in 97 frames (100.00%).
Processed 106 frames.
Detected faces in 106 frames (100.00%).
Processed 94 frames.
Detected faces in 94 frames (100.00%).
Processed 108 frames.
Detected faces in 108 frames (100.00%).
Processed 95 frames.
Detected faces in 95 frames (100.00%).
Processed 103 frames.
Detected faces in 103 frames (100.00%).
Processed 100 frames.
Detected faces in 100 frames (100.00%).
Processed 112 frames.
Detected faces in 112 frames (100.00%).
Processed 114 frames.

Detected faces in 114 frames (100.00%).
Processed 104 frames.
Detected faces in 104 frames (100.00%).
Processed 100 frames.
Detected faces in 100 frames (100.00%).
Processed 104 frames.
Detected faces in 104 frames (100.00%).
Processed 115 frames.
Detected faces in 115 frames (100.00%).
Processed 109 frames.
Detected faces in 109 frames (100.00%).
Processed 114 frames.
Detected faces in 114 frames (100.00%).
Processed 111 frames.
Detected faces in 111 frames (100.00%).
Processed 99 frames.
Detected faces in 99 frames (100.00%).
Processed 108 frames.
Detected faces in 108 frames (100.00%).
Processed 105 frames.
Detected faces in 105 frames (100.00%).
Processed 101 frames.
Detected faces in 101 frames (100.00%).
Processed 109 frames.
Detected faces in 109 frames (100.00%).
Processed 107 frames.
Detected faces in 107 frames (100.00%).
Processed 114 frames.
Detected faces in 114 frames (100.00%).
Processed 109 frames.
Detected faces in 109 frames (100.00%).
Processed 95 frames.
Detected faces in 95 frames (100.00%).
Processed 115 frames.
Detected faces in 115 frames (100.00%).
Processed 105 frames.
Detected faces in 105 frames (100.00%).
Processed 96 frames.
Detected faces in 96 frames (100.00%).
Processed 99 frames.
Detected faces in 99 frames (100.00%).
Processed 102 frames.
Detected faces in 102 frames (100.00%).
Processed 112 frames.
Detected faces in 112 frames (100.00%).
Processed 104 frames.
Detected faces in 104 frames (100.00%).
Processed 106 frames.
Detected faces in 106 frames (100.00%).
Processed 96 frames.
Detected faces in 96 frames (100.00%).
Processed 107 frames.
Detected faces in 107 frames (100.00%).

Processed 119 frames.
Detected faces in 119 frames (100.00%).
Processed 103 frames.
Detected faces in 103 frames (100.00%).
Processed 112 frames.
Detected faces in 112 frames (100.00%).
Processed 109 frames.
Detected faces in 109 frames (100.00%).
Processed 114 frames.
Detected faces in 114 frames (100.00%).
Processed 115 frames.
Detected faces in 115 frames (100.00%).
Processed 108 frames.
Detected faces in 108 frames (100.00%).
Processed 101 frames.
Detected faces in 101 frames (100.00%).
Processed 98 frames.
Detected faces in 98 frames (100.00%).
Processed 106 frames.
Detected faces in 106 frames (100.00%).
Processed 126 frames.
Detected faces in 126 frames (100.00%).
Processed 97 frames.
Detected faces in 97 frames (100.00%).
Processed 99 frames.
Detected faces in 99 frames (100.00%).
Processed 100 frames.
Detected faces in 100 frames (100.00%).
Processed 109 frames.
Detected faces in 109 frames (100.00%).
Processed 107 frames.
Detected faces in 107 frames (100.00%).
Processed 111 frames.
Detected faces in 111 frames (100.00%).
Processed 104 frames.
Detected faces in 104 frames (100.00%).
Processed 107 frames.
Detected faces in 107 frames (100.00%).
Processed 109 frames.
Detected faces in 109 frames (100.00%).
Processed 107 frames.
Detected faces in 107 frames (100.00%).
Processed 102 frames.
Detected faces in 102 frames (100.00%).
Processed 108 frames.
Detected faces in 108 frames (100.00%).
Processed 129 frames.
Detected faces in 129 frames (100.00%).
Processed 108 frames.
Detected faces in 108 frames (100.00%).
Processed 108 frames.
Detected faces in 108 frames (100.00%).
Processed 123 frames.

Detected faces in 123 frames (100.00%).
Processed 106 frames.
Detected faces in 106 frames (100.00%).
Processed 107 frames.
Detected faces in 107 frames (100.00%).
Processed 102 frames.
Detected faces in 102 frames (100.00%).
Processed 109 frames.
Detected faces in 109 frames (100.00%).
Processed 107 frames.
Detected faces in 107 frames (100.00%).
Processed 108 frames.
Detected faces in 108 frames (100.00%).
Processed 116 frames.
Detected faces in 116 frames (100.00%).
Processed 102 frames.
Detected faces in 102 frames (100.00%).
Processed 104 frames.
Detected faces in 104 frames (100.00%).
Processed 108 frames.
Detected faces in 108 frames (100.00%).
Processed 107 frames.
Detected faces in 107 frames (100.00%).
Processed 106 frames.
Detected faces in 106 frames (100.00%).
Processed 107 frames.
Detected faces in 107 frames (100.00%).
Processed 98 frames.
Detected faces in 98 frames (100.00%).
Processed 109 frames.
Detected faces in 109 frames (100.00%).
Processed 104 frames.
Detected faces in 104 frames (100.00%).
Processed 109 frames.
Detected faces in 109 frames (100.00%).
Processed 109 frames.
Detected faces in 109 frames (100.00%).
Processed 105 frames.
Detected faces in 105 frames (100.00%).
Processed 138 frames.
Detected faces in 138 frames (100.00%).
Processed 111 frames.
Detected faces in 111 frames (100.00%).
Processed 116 frames.
Detected faces in 116 frames (100.00%).
Processed 119 frames.
Detected faces in 119 frames (100.00%).
Processed 113 frames.
Detected faces in 113 frames (100.00%).
Processed 112 frames.
Detected faces in 112 frames (100.00%).
Processed 113 frames.
Detected faces in 113 frames (100.00%).

Processed 110 frames.
Detected faces in 110 frames (100.00%).
Processed 106 frames.
Detected faces in 106 frames (100.00%).
Processed 125 frames.
Detected faces in 125 frames (100.00%).
Processed 118 frames.
Detected faces in 118 frames (100.00%).
Processed 113 frames.
Detected faces in 113 frames (100.00%).
Processed 117 frames.
Detected faces in 117 frames (100.00%).
Processed 105 frames.
Detected faces in 105 frames (100.00%).
Processed 100 frames.
Detected faces in 100 frames (100.00%).
Processed 115 frames.
Detected faces in 115 frames (100.00%).
Processed 100 frames.
Detected faces in 100 frames (100.00%).
Processed 115 frames.
Detected faces in 115 frames (100.00%).
Processed 97 frames.
Detected faces in 97 frames (100.00%).
Processed 112 frames.
Detected faces in 112 frames (100.00%).
Processed 109 frames.
Detected faces in 109 frames (100.00%).
Processed 114 frames.
Detected faces in 114 frames (100.00%).
Processed 119 frames.
Detected faces in 119 frames (100.00%).
Processed 113 frames.
Detected faces in 113 frames (100.00%).
Processed 125 frames.
Detected faces in 125 frames (100.00%).
Processed 112 frames.
Detected faces in 112 frames (100.00%).
Processed 128 frames.
Detected faces in 128 frames (100.00%).
Processed 132 frames.
Detected faces in 132 frames (100.00%).
Processed 110 frames.
Detected faces in 110 frames (100.00%).
Processed 113 frames.
Detected faces in 113 frames (100.00%).
Processed 115 frames.
Detected faces in 115 frames (100.00%).
Processed 113 frames.
Detected faces in 113 frames (100.00%).
Processed 98 frames.
Detected faces in 98 frames (100.00%).
Processed 122 frames.

Detected faces in 122 frames (100.00%).
Processed 115 frames.
Detected faces in 115 frames (100.00%).
Processed 118 frames.
Detected faces in 118 frames (100.00%).
Processed 116 frames.
Detected faces in 116 frames (100.00%).
Processed 107 frames.
Detected faces in 107 frames (100.00%).
Processed 108 frames.
Detected faces in 108 frames (100.00%).
Processed 112 frames.
Detected faces in 112 frames (100.00%).
Processed 99 frames.
Detected faces in 99 frames (100.00%).
Processed 109 frames.
Detected faces in 109 frames (100.00%).
Processed 101 frames.
Detected faces in 101 frames (100.00%).
Processed 114 frames.
Detected faces in 114 frames (100.00%).
Processed 110 frames.
Detected faces in 110 frames (100.00%).
Processed 115 frames.
Detected faces in 115 frames (100.00%).
Processed 110 frames.
Detected faces in 110 frames (100.00%).
Processed 127 frames.
Detected faces in 127 frames (100.00%).
Processed 128 frames.
Detected faces in 128 frames (100.00%).
Processed 109 frames.
Detected faces in 109 frames (100.00%).
Processed 131 frames.
Detected faces in 131 frames (100.00%).
Processed 137 frames.
Detected faces in 137 frames (100.00%).
Processed 101 frames.
Detected faces in 101 frames (100.00%).
Processed 106 frames.
Detected faces in 106 frames (100.00%).
Processed 97 frames.
Detected faces in 97 frames (100.00%).
Processed 97 frames.
Detected faces in 97 frames (100.00%).
Processed 112 frames.
Detected faces in 112 frames (100.00%).
Processed 96 frames.
Detected faces in 96 frames (100.00%).
Processed 104 frames.
Detected faces in 104 frames (100.00%).
Processed 111 frames.
Detected faces in 111 frames (100.00%).

Processed 91 frames.
Detected faces in 91 frames (100.00%).
Processed 97 frames.
Detected faces in 97 frames (100.00%).
Processed 92 frames.
Detected faces in 92 frames (100.00%).
Processed 105 frames.
Detected faces in 105 frames (100.00%).
Processed 108 frames.
Detected faces in 108 frames (100.00%).
Processed 108 frames.
Detected faces in 108 frames (100.00%).
Processed 106 frames.
Detected faces in 106 frames (100.00%).
Processed 101 frames.
Detected faces in 101 frames (100.00%).
Processed 101 frames.
Detected faces in 101 frames (100.00%).
Processed 111 frames.
Detected faces in 111 frames (100.00%).
Processed 97 frames.
Detected faces in 97 frames (100.00%).
Processed 105 frames.
Detected faces in 105 frames (100.00%).
Processed 106 frames.
Detected faces in 106 frames (100.00%).
Processed 98 frames.
Detected faces in 98 frames (100.00%).
Processed 99 frames.
Detected faces in 99 frames (100.00%).
Processed 97 frames.
Detected faces in 97 frames (100.00%).
Processed 106 frames.
Detected faces in 106 frames (100.00%).
Processed 101 frames.
Detected faces in 101 frames (100.00%).
Processed 100 frames.
Detected faces in 100 frames (100.00%).
Processed 110 frames.
Detected faces in 110 frames (100.00%).
Processed 96 frames.
Detected faces in 96 frames (100.00%).
Processed 106 frames.
Detected faces in 106 frames (100.00%).
Processed 114 frames.
Detected faces in 114 frames (100.00%).
Processed 99 frames.
Detected faces in 99 frames (100.00%).
Processed 102 frames.
Detected faces in 102 frames (100.00%).
Processed 94 frames.
Detected faces in 94 frames (100.00%).
Processed 108 frames.

Detected faces in 108 frames (100.00%).
Processed 94 frames.
Detected faces in 94 frames (100.00%).
Processed 106 frames.
Detected faces in 106 frames (100.00%).
Processed 102 frames.
Detected faces in 102 frames (100.00%).
Processed 103 frames.
Detected faces in 103 frames (100.00%).
Processed 108 frames.
Detected faces in 108 frames (100.00%).
Processed 105 frames.
Detected faces in 105 frames (100.00%).
Processed 105 frames.
Detected faces in 105 frames (100.00%).
Processed 107 frames.
Detected faces in 107 frames (100.00%).
Processed 104 frames.
Detected faces in 104 frames (100.00%).
Processed 107 frames.
Detected faces in 107 frames (100.00%).
Processed 97 frames.
Detected faces in 97 frames (100.00%).
Processed 104 frames.
Detected faces in 104 frames (100.00%).
Processed 102 frames.
Detected faces in 102 frames (100.00%).
Processed 112 frames.
Detected faces in 112 frames (100.00%).
Processed 98 frames.
Detected faces in 98 frames (100.00%).
Processed 110 frames.
Detected faces in 110 frames (100.00%).
Processed 124 frames.
Detected faces in 124 frames (100.00%).
Processed 96 frames.
Detected faces in 96 frames (100.00%).
Processed 116 frames.
Detected faces in 116 frames (100.00%).
Processed 115 frames.
Detected faces in 115 frames (100.00%).
Processed 98 frames.
Detected faces in 98 frames (100.00%).
Processed 105 frames.
Detected faces in 105 frames (100.00%).
Processed 92 frames.
Detected faces in 92 frames (100.00%).
Processed 110 frames.
Detected faces in 110 frames (100.00%).
Processed 100 frames.
Detected faces in 100 frames (100.00%).
Processed 118 frames.
Detected faces in 118 frames (100.00%).

Processed 100 frames.
Detected faces in 100 frames (100.00%).
Processed 106 frames.
Detected faces in 106 frames (100.00%).
Processed 105 frames.
Detected faces in 105 frames (100.00%).
Processed 109 frames.
Detected faces in 109 frames (100.00%).
Processed 122 frames.
Detected faces in 122 frames (100.00%).
Processed 112 frames.
Detected faces in 112 frames (100.00%).
Processed 109 frames.
Detected faces in 109 frames (100.00%).
Processed 107 frames.
Detected faces in 107 frames (100.00%).
Processed 105 frames.
Detected faces in 105 frames (100.00%).
Processed 118 frames.
Detected faces in 118 frames (100.00%).
Processed 106 frames.
Detected faces in 106 frames (100.00%).
Processed 113 frames.
Detected faces in 113 frames (100.00%).
Processed 108 frames.
Detected faces in 108 frames (100.00%).
Processed 107 frames.
Detected faces in 107 frames (100.00%).
Processed 123 frames.
Detected faces in 123 frames (100.00%).
Processed 100 frames.
Detected faces in 100 frames (100.00%).
Processed 109 frames.
Detected faces in 109 frames (100.00%).
Processed 111 frames.
Detected faces in 111 frames (100.00%).
Processed 97 frames.
Detected faces in 97 frames (100.00%).
Processed 102 frames.
Detected faces in 102 frames (100.00%).
Processed 93 frames.
Detected faces in 93 frames (100.00%).
Processed 111 frames.
Detected faces in 111 frames (100.00%).
Processed 99 frames.
Detected faces in 99 frames (100.00%).
Processed 110 frames.
Detected faces in 110 frames (100.00%).
Processed 98 frames.
Detected faces in 98 frames (100.00%).
Processed 109 frames.
Detected faces in 109 frames (100.00%).
Processed 106 frames.

```

Detected faces in 106 frames (100.00%).
Processed 138 frames.
Detected faces in 138 frames (100.00%).
Processed 97 frames.
Detected faces in 97 frames (100.00%).
Processed 109 frames.
Detected faces in 109 frames (100.00%).
Processed 112 frames.
Detected faces in 112 frames (100.00%).
Processed 101 frames.
Detected faces in 101 frames (100.00%).
Processed 119 frames.
Detected faces in 119 frames (100.00%).

```

This prepares the variable-length video sequences (in X) so they can be batched together and fed efficiently into your PyTorch model (CNN+LSTM). Neural networks require all inputs in a batch to have the same shape, but your videos have different lengths (different number of frames). Padding solves this.

```

In [97]: # --- Pad Sequences ---
feature_dim = len(X[0][0]) if X and X[0] else 0
X_tensors = [torch.tensor(seq, dtype=torch.float32) for seq in X]
X_padded = pad_sequence(X_tensors, batch_first=True)
y = torch.tensor(y, dtype=torch.long)
lengths = np.array(lengths)

```

```

C:\Users\hp\AppData\Local\Temp\ipykernel_18876\1211500649.py:5:
UserWarning: To copy construct from a tensor, it is recommended to
use sourceTensor.clone().detach() or
sourceTensor.clone().detach().requires_grad_(True), rather than
torch.tensor(sourceTensor).
  y = torch.tensor(y, dtype=torch.long)

```

```
In [98]: # --- Split Train/Test ---
num_samples = len(X)
indices = np.arange(num_samples)
np.random.shuffle(indices)
train_size = int(0.8 * num_samples)
train_idx = indices[:train_size]
test_idx = indices[train_size:]

X_train = X_padded[train_idx]
y_train = y[train_idx]
train_lengths = lengths[train_idx]

X_test = X_padded[test_idx]
y_test = y[test_idx]
test_lengths = lengths[test_idx]

# --- Compute Class Weights for Imbalance ---
class_weights = compute_class_weight('balanced',
classes=np.unique(y.numpy()), y=y.numpy())
class_weights = torch.tensor(class_weights, dtype=torch.float32)
print(f"Class weights: {class_weights}")

Class weights: tensor([0.9362, 0.9362, 0.9362, 0.9362, 0.9362,
1.9130, 0.9362, 0.9362])
```

```
In [99]: # Convert lengths to torch tensors
train_lengths = torch.tensor(train_lengths, dtype=torch.long)
test_lengths = torch.tensor(test_lengths, dtype=torch.long)
```

Normalization means scaling features so they have similar value ranges

With normalization:

Faster convergence

Stable gradients

Better accuracy

```

In [100]: # --- Normalize Features (FIXED) ---
# Step 1: Fit scaler only on real frames (exclude padding)
# X_train is (num_samples, max_seq_len, feature_dim)
train_numpy = X_train.numpy() # shape: (N, T, F)
real_frames_list = []
for i in range(len(train_numpy)):
    length = train_lengths[i].item()
    real_frames_list.append(train_numpy[i, :length, :]) # take only actual
frames

# Concatenate all real frames → (total_real_frames, feature_dim)
all_train_frames = np.concatenate(real_frames_list, axis=0)

scaler = StandardScaler()
scaler.fit(all_train_frames)

```

```

Out[100]: StandardScaler ⓘ ?
StandardScaler()

```

```

In [ ]: # Step 2: Apply transform to full padded sequences
#normalize every frame in the padded input using statistics learned only
from real frames, without changing the sequence shape.
def normalize_padded_sequences(padded_tensor):
    """Apply scaler to a padded 3D tensor (batch, seq, feat)"""
    numpy_3d = padded_tensor.numpy()
    N, T, F = numpy_3d.shape
    # Reshape to 2D: (N*T, F)
    flat_2d = numpy_3d.reshape(-1, F)
    # Transform
    transformed_2d = scaler.transform(flat_2d)
    # Reshape back to 3D
    transformed_3d = transformed_2d.reshape(N, T, F)
    return torch.tensor(transformed_3d, dtype=torch.float32)

X_train_norm = normalize_padded_sequences(X_train)
X_test_norm = normalize_padded_sequences(X_test)

print(f"Normalization complete. Feature dim: {X_train_norm.shape[2]}")

```

Normalization complete. Feature dim: 166

F contains stateless functions like `relu`, `softmax`, `cross_entropy`
Used instead of `nn.ReLU()` when you don't need a layer object

✦ Here it's used for ReLU activation inside the residual block

Residual Convolutional Block

`in_channels` → number of input feature channels

`out_channels` → number of filters to learn

Conv1D slides over time

`kernel_size=3` → looks at 3 consecutive frame

`padding=1` → preserves temporal length

✦ This learns local motion patterns (micro-expressions).

BatchNormD

Normalizes activations

Stabilizes training

Helps deeper networks converge

Shortcut (Residual connection)

solve:

Vanishing gradients

Information loss

`forward()` method

Input `x` goes through convolution

Batch normalization applied

Shortcut path added

ReLU activation applied

Advanced CNN + LSTM + Attention Model

CNN → local temporal patterns

LSTM → long-term temporal dependencies

Attention → focus on important frames

CNN Feature Extractor

Sequential

Extract short-term temporal patterns before LSTM.

First Residual Block

Expands feature space → richer representation

MaxPooling halves time dimension

Second Block

Compresses features

Focuses on stronger patterns

Third Block

Further temporal abstraction

Final Block

Refines features

No pooling → preserve information

Bi-Directional LSTM

Why LSTM

CNN sees local patterns

LSTM sees sequence context

Why Bi-directional

Facial expressions depend on:

What happened before

What happens after

Multi-Head Self-Attention

Instead of treating all frames equally:

Learns which frames matter most

Why multi-head

Each head focuses on:

Eyes

Mouth

Sudden motion

Subtle changes

📌 Extremely powerful for expression recognition.

Fully Connected Classifier

```
self.fc = nn.Sequential(
```

Purpose

Convert high-level temporal features → emotion classes.

Forward Pass

```

In [102]: import torch.nn.functional as F
# --- Residual Convolutional Block ---
class ResidualConvBlock(nn.Module):
    def __init__(self, in_channels, out_channels):
        super().__init__()
        self.conv = nn.Conv1d(in_channels, out_channels, kernel_size=3,
padding=1)
        self.bn = nn.BatchNorm1d(out_channels)
        self.shortcut = nn.Conv1d(in_channels, out_channels, kernel_size=1)
    if in_channels != out_channels else nn.Identity()

    def forward(self, x):
        return F.relu(self.bn(self.conv(x)) + self.shortcut(x))

# --- Advanced CNN+LSTM Model with Residual Connections and Multi-Head
Attention ---
class AdvancedCNNLSTM(nn.Module):
    def __init__(self, input_dim, hidden_dim, num_classes, num_layers=2):
        super().__init__()

        # Deeper CNN with Residual Connections
        self.cnn = nn.Sequential(
            ResidualConvBlock(input_dim, 256),
            nn.MaxPool1d(2),
            ResidualConvBlock(256, 128),
            nn.MaxPool1d(2),
            ResidualConvBlock(128, 64),
            nn.MaxPool1d(2),
            ResidualConvBlock(64, 64)
        )

        # Bi-LSTM to capture temporal dependencies
        self.lstm = nn.LSTM(64, hidden_dim, batch_first=True,
                                num_layers=num_layers, bidirectional=True,
dropout=0.3)

        # Multi-Head Self-Attention Layer
        self.attention = nn.MultiheadAttention(
            embed_dim=hidden_dim * 2,
            num_heads=8,
            batch_first=True,
            dropout=0.1
        )

        self.fc = nn.Sequential(
            nn.Linear(hidden_dim * 2, 512),
            nn.ReLU(),
            nn.Dropout(0.5),
            nn.Linear(512, 256),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(256, num_classes)
        )

```

```
def forward(self, x, lengths):
    # x shape: (B, T, F) where B=batch, T=time, F=features
    x = x.permute(0, 2, 1) # (B, F, T)
    x = self.cnn(x) # (B, 64, T//8) due to 3 MaxPool1d layers
    x = x.permute(0, 2, 1) # (B, T//8, 64)

    # Adjust lengths for pooling (divide by 8 due to three MaxPool1d
layers)
    adj_len = (lengths.cpu() // 8).clamp(min=1)
    packed = pack_padded_sequence(x, adj_len, batch_first=True,
enforce_sorted=False)
    lstm_out, _ = self.lstm(packed)
    lstm_out, _ = nn.utils.rnn.pad_packed_sequence(lstm_out,
batch_first=True)
    # lstm_out: (B, T//8, hidden*2)

    # Multi-head attention
    attn_out, _ = self.attention(lstm_out, lstm_out, lstm_out)

    # Global Average Pooling over the temporal dimension
    out = torch.mean(attn_out, dim=1) # (B, hidden*2)
    return self.fc(out)
```

CNN + LSTM

Why CNN first?

LSTMs are good at long-term dependencies, but:

They are weak at extracting local patterns

CNNs excel at detecting local correlations across time

So CNN acts as a feature encoder before LSTM.

Why residual connections?

Prevent vanishing gradients

Allow deeper CNNs

Preserve original signal while learning refinements

✚ Especially important for facial dynamics, where subtle movements matter.

What LSTM does here

Processes sequence order

Learns:

Onset → peak → offset of expressions

Temporal consistency across frames

Why LSTM after CNN?

CNN extracts what happens

LSTM learns when it happens

Facial expressions depend on both past and future frames, so bidirectionality improves accuracy.

`hidden_dim = 512`

This is the LSTM hidden state size

`num_classes = len(labels)`

Number of emotion categories

`class ExpressionDataset(Dataset)`

model needs three things per sample:

Feature sequence

Label

Sequence length (for packing LSTM)

```

In [103]: # --- Initialize Advanced Model ---
hidden_dim = 512
num_classes = len(labels)
model = AdvancedCNLSTM(feature_dim, hidden_dim, num_classes, num_layers=2)
epochs=200

# Use class-weighted loss with label smoothing to prevent overconfidence
criterion = nn.CrossEntropyLoss(weight=class_weights, label_smoothing=0.1)
optimizer = optim.AdamW(model.parameters(), lr=0.001, weight_decay=1e-4)

# One-cycle learning rate scheduler for better convergence
from torch.utils.data import Dataset

class ExpressionDataset(Dataset):
    def __init__(self, features, labels, lengths):
        self.features = features
        self.labels = labels
        self.lengths = lengths

    def __len__(self):
        return len(self.labels)

    def __getitem__(self, idx):
        return self.features[idx], self.labels[idx], self.lengths[idx]

# Now you can safely create your DataLoader
train_loader = torch.utils.data.DataLoader(
    ExpressionDataset(X_train, y_train, train_lengths),
    batch_size=32,
    sampler = weighted_sampler,
)

# Calculate steps and initialize scheduler
steps_per_epoch = len(train_loader)

optimizer = optim.AdamW(model.parameters(), lr=0.001, weight_decay=1e-4)

total_steps = epochs * steps_per_epoch

scheduler = optim.lr_scheduler.OneCycleLR(
    optimizer,
    max_lr=0.001,
    anneal_strategy='cos',
    total_steps=total_steps
)

```

```

In
[104]: from torch.utils.data import WeightedRandomSampler, TensorDataset,
        DataLoader
        import time
        import torch.nn.functional as F
        # --- Create DataLoader with WeightedRandomSampler for Class Balance ---
        # Compute sample weights inversely proportional to class frequency
        sample_weights = []
        for idx in y_train:
            # Weight = 1 / (class_weight * class_count)
            sample_weights.append(1.0 / (class_weights[idx].item() * (y_train ==
idx).sum().item()))

        sample_weights = torch.tensor(sample_weights, dtype=torch.float32)
        weighted_sampler = WeightedRandomSampler(
            weights=sample_weights,
            num_samples=len(sample_weights),
            replacement=True
        )

        # Create datasets and dataloaders
        train_dataset = TensorDataset(X_train_norm, train_lengths, y_train)
        train_loader = DataLoader(
            train_dataset,
            batch_size=32,
            sampler=weighted_sampler,
            drop_last=True
        )

        test_dataset = TensorDataset(X_test_norm, test_lengths, y_test)
        test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False)

        # scheduler._init_with_optimizer(optimizer, 200, len(train_loader))

        # --- Training Loop with Enhanced Early Stopping ---
        epochs = 200
        best_acc = 0.0
        best_loss = float('inf')
        patience = 40
        counter = 0
        training_history = {'loss': [], 'val_loss': [], 'val_acc': []}

        start_time = time.time()

        for epoch in range(epochs):
            model.train()
            total_loss = 0.0
            batch_count = 0

            for batch_X, batch_lengths, batch_y in train_loader:
                optimizer.zero_grad()
                outputs = model(batch_X, batch_lengths)
                loss = criterion(outputs, batch_y)
                loss.backward()

```

```

        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0) #
Gradient clipping
        optimizer.step()
        if scheduler.last_epoch < scheduler.total_steps:
            scheduler.step()

        total_loss += loss.item()
        batch_count += 1

    avg_loss = total_loss / batch_count
    training_history['loss'].append(avg_loss)

# Validation
model.eval()
val_total_loss = 0.0
correct = 0
total = 0

with torch.no_grad():
    for val_X, val_lengths, val_y in test_loader:
        val_outputs = model(val_X, val_lengths)
        val_loss = criterion(val_outputs, val_y)
        val_total_loss += val_loss.item()

        val_pred = val_outputs.argmax(dim=1)
        correct += (val_pred == val_y).sum().item()
        total += val_y.size(0)

val_loss = val_total_loss / len(test_loader)
val_acc = correct / total

training_history['val_loss'].append(val_loss)
training_history['val_acc'].append(val_acc)

current_lr = optimizer.param_groups[0]['lr']
print(f"Epoch {epoch+1:3d}/{epochs} | Loss: {avg_loss:.4f} | Val Loss:
{val_loss:.4f} | Val Acc: {val_acc*100:.2f}% | LR: {current_lr:.6f}")

# Save best model based on validation accuracy
if val_acc > best_acc or (val_acc == best_acc and val_loss <
best_loss):
    best_acc = val_acc
    best_loss = val_loss
    counter = 0
    torch.save(model.state_dict(),
'landmark_facial_expression_model.pth')
    print(f" >>> Model saved! Best Acc: {best_acc*100:.2f}%")
else:
    counter += 1

# Early stopping with patience
if counter >= patience:
    print(f"\nEarly stopping triggered after {epoch+1} epochs

```



```
(patience: {patience}))"
    print(f"Best Validation Accuracy: {best_acc*100:.2f}%")
    break

training_time = time.time() - start_time
print(f"\nTraining completed in {training_time/60:.2f} minutes")
```

```
Epoch 1/200 | Loss: 2.0836 | Val Loss: 2.0936 | Val Acc: 9.93% |
LR: 0.000041
>>> Model saved! Best Acc: 9.93%
Epoch 2/200 | Loss: 2.0833 | Val Loss: 2.0858 | Val Acc: 12.06% |
LR: 0.000042
>>> Model saved! Best Acc: 12.06%
Epoch 3/200 | Loss: 2.0729 | Val Loss: 2.0545 | Val Acc: 24.11% |
LR: 0.000045
>>> Model saved! Best Acc: 24.11%
Epoch 4/200 | Loss: 2.0231 | Val Loss: 1.9709 | Val Acc: 20.57% |
LR: 0.000049
Epoch 5/200 | Loss: 1.9906 | Val Loss: 1.8581 | Val Acc: 21.99% |
LR: 0.000055
Epoch 6/200 | Loss: 1.8702 | Val Loss: 1.7979 | Val Acc: 21.28% |
LR: 0.000061
Epoch 7/200 | Loss: 1.8711 | Val Loss: 1.7642 | Val Acc: 28.37% |
LR: 0.000069
>>> Model saved! Best Acc: 28.37%
Epoch 8/200 | Loss: 1.7329 | Val Loss: 1.6102 | Val Acc: 30.50% |
LR: 0.000077
>>> Model saved! Best Acc: 30.50%
Epoch 9/200 | Loss: 1.6295 | Val Loss: 1.5854 | Val Acc: 35.46% |
LR: 0.000087
>>> Model saved! Best Acc: 35.46%
Epoch 10/200 | Loss: 1.6275 | Val Loss: 1.5445 | Val Acc: 39.72% |
LR: 0.000098
>>> Model saved! Best Acc: 39.72%
Epoch 11/200 | Loss: 1.5610 | Val Loss: 1.4464 | Val Acc: 46.81% |
LR: 0.000109
>>> Model saved! Best Acc: 46.81%
Epoch 12/200 | Loss: 1.5159 | Val Loss: 1.5990 | Val Acc: 39.72% |
LR: 0.000122
Epoch 13/200 | Loss: 1.4818 | Val Loss: 1.5467 | Val Acc: 43.26% |
LR: 0.000136
Epoch 14/200 | Loss: 1.5501 | Val Loss: 1.3760 | Val Acc: 48.23% |
LR: 0.000151
>>> Model saved! Best Acc: 48.23%
Epoch 15/200 | Loss: 1.4409 | Val Loss: 1.4442 | Val Acc: 45.39% |
LR: 0.000166
Epoch 16/200 | Loss: 1.3642 | Val Loss: 1.4504 | Val Acc: 43.97% |
LR: 0.000183
Epoch 17/200 | Loss: 1.5340 | Val Loss: 1.4197 | Val Acc: 47.52% |
LR: 0.000200
Epoch 18/200 | Loss: 1.4418 | Val Loss: 1.4056 | Val Acc: 48.94% |
LR: 0.000218
>>> Model saved! Best Acc: 48.94%
```

```
Epoch 19/200 | Loss: 1.4014 | Val Loss: 1.3599 | Val Acc: 54.61% |  
LR: 0.000237  
>>> Model saved! Best Acc: 54.61%  
Epoch 20/200 | Loss: 1.3069 | Val Loss: 1.3806 | Val Acc: 57.45% |  
LR: 0.000257  
>>> Model saved! Best Acc: 57.45%  
Epoch 21/200 | Loss: 1.3966 | Val Loss: 1.4469 | Val Acc: 51.06% |  
LR: 0.000277  
Epoch 22/200 | Loss: 1.2626 | Val Loss: 1.4415 | Val Acc: 53.19% |  
LR: 0.000298  
Epoch 23/200 | Loss: 1.2178 | Val Loss: 1.3976 | Val Acc: 53.19% |  
LR: 0.000319  
Epoch 24/200 | Loss: 1.3178 | Val Loss: 1.3971 | Val Acc: 47.52% |  
LR: 0.000341  
Epoch 25/200 | Loss: 1.3529 | Val Loss: 1.4852 | Val Acc: 50.35% |  
LR: 0.000363  
Epoch 26/200 | Loss: 1.2940 | Val Loss: 1.3067 | Val Acc: 60.28% |  
LR: 0.000386  
>>> Model saved! Best Acc: 60.28%  
Epoch 27/200 | Loss: 1.1923 | Val Loss: 1.5615 | Val Acc: 53.90% |  
LR: 0.000409  
Epoch 28/200 | Loss: 1.3353 | Val Loss: 1.3406 | Val Acc: 58.16% |  
LR: 0.000432  
Epoch 29/200 | Loss: 1.2346 | Val Loss: 1.5375 | Val Acc: 53.19% |  
LR: 0.000455  
Epoch 30/200 | Loss: 1.2703 | Val Loss: 1.2188 | Val Acc: 67.38% |  
LR: 0.000479  
>>> Model saved! Best Acc: 67.38%  
Epoch 31/200 | Loss: 1.2169 | Val Loss: 1.2813 | Val Acc: 62.41% |  
LR: 0.000503  
Epoch 32/200 | Loss: 1.2649 | Val Loss: 1.2926 | Val Acc: 61.70% |  
LR: 0.000526  
Epoch 33/200 | Loss: 1.3428 | Val Loss: 1.2958 | Val Acc: 67.38% |  
LR: 0.000550  
Epoch 34/200 | Loss: 1.2413 | Val Loss: 1.3106 | Val Acc: 63.83% |  
LR: 0.000574  
Epoch 35/200 | Loss: 1.1693 | Val Loss: 1.4098 | Val Acc: 53.90% |  
LR: 0.000597  
Epoch 36/200 | Loss: 1.2335 | Val Loss: 1.4436 | Val Acc: 58.87% |  
LR: 0.000621  
Epoch 37/200 | Loss: 1.1599 | Val Loss: 1.4216 | Val Acc: 59.57% |  
LR: 0.000644  
Epoch 38/200 | Loss: 1.1292 | Val Loss: 1.1957 | Val Acc: 68.79% |  
LR: 0.000666  
>>> Model saved! Best Acc: 68.79%  
Epoch 39/200 | Loss: 1.1534 | Val Loss: 1.2713 | Val Acc: 63.83% |  
LR: 0.000689  
Epoch 40/200 | Loss: 1.0624 | Val Loss: 1.1450 | Val Acc: 68.79% |  
LR: 0.000711  
>>> Model saved! Best Acc: 68.79%  
Epoch 41/200 | Loss: 1.0946 | Val Loss: 1.4118 | Val Acc: 59.57% |  
LR: 0.000732  
Epoch 42/200 | Loss: 1.2038 | Val Loss: 1.4114 | Val Acc: 64.54% |
```

```
LR: 0.000754
Epoch 43/200 | Loss: 1.1953 | Val Loss: 1.3144 | Val Acc: 65.25% |
LR: 0.000774
Epoch 44/200 | Loss: 1.0569 | Val Loss: 1.2601 | Val Acc: 63.12% |
LR: 0.000794
Epoch 45/200 | Loss: 1.0389 | Val Loss: 1.2464 | Val Acc: 63.83% |
LR: 0.000813
Epoch 46/200 | Loss: 0.9972 | Val Loss: 1.2763 | Val Acc: 62.41% |
LR: 0.000831
Epoch 47/200 | Loss: 0.9807 | Val Loss: 1.0458 | Val Acc: 74.47% |
LR: 0.000849
>>> Model saved! Best Acc: 74.47%
Epoch 48/200 | Loss: 0.9665 | Val Loss: 1.2721 | Val Acc: 62.41% |
LR: 0.000866
Epoch 49/200 | Loss: 0.9705 | Val Loss: 1.0965 | Val Acc: 72.34% |
LR: 0.000882
Epoch 50/200 | Loss: 0.9654 | Val Loss: 1.1820 | Val Acc: 63.83% |
LR: 0.000897
Epoch 51/200 | Loss: 0.9614 | Val Loss: 1.2415 | Val Acc: 65.25% |
LR: 0.000911
Epoch 52/200 | Loss: 1.0033 | Val Loss: 1.2767 | Val Acc: 72.34% |
LR: 0.000925
Epoch 53/200 | Loss: 0.9488 | Val Loss: 1.0824 | Val Acc: 78.01% |
LR: 0.000937
>>> Model saved! Best Acc: 78.01%
Epoch 54/200 | Loss: 0.8596 | Val Loss: 1.1106 | Val Acc: 67.38% |
LR: 0.000948
Epoch 55/200 | Loss: 0.9483 | Val Loss: 1.0414 | Val Acc: 75.18% |
LR: 0.000958
Epoch 56/200 | Loss: 0.8990 | Val Loss: 1.0819 | Val Acc: 77.30% |
LR: 0.000968
Epoch 57/200 | Loss: 0.8256 | Val Loss: 1.5422 | Val Acc: 70.21% |
LR: 0.000976
Epoch 58/200 | Loss: 0.9151 | Val Loss: 1.0374 | Val Acc: 76.60% |
LR: 0.000983
Epoch 59/200 | Loss: 0.8658 | Val Loss: 1.1916 | Val Acc: 75.89% |
LR: 0.000988
Epoch 60/200 | Loss: 0.8480 | Val Loss: 1.3623 | Val Acc: 72.34% |
LR: 0.000993
Epoch 61/200 | Loss: 0.8579 | Val Loss: 1.0611 | Val Acc: 76.60% |
LR: 0.000996
Epoch 62/200 | Loss: 0.7322 | Val Loss: 1.1416 | Val Acc: 75.89% |
LR: 0.000999
Epoch 63/200 | Loss: 0.9102 | Val Loss: 1.4224 | Val Acc: 66.67% |
LR: 0.001000
Epoch 64/200 | Loss: 0.9249 | Val Loss: 1.0955 | Val Acc: 75.89% |
LR: 0.001000
Epoch 65/200 | Loss: 0.7689 | Val Loss: 1.0915 | Val Acc: 77.30% |
LR: 0.001000
Epoch 66/200 | Loss: 0.8030 | Val Loss: 1.0266 | Val Acc: 77.30% |
LR: 0.000999
Epoch 67/200 | Loss: 0.7809 | Val Loss: 1.2072 | Val Acc: 74.47% |
LR: 0.000999
```

```
Epoch 68/200 | Loss: 0.8045 | Val Loss: 1.0492 | Val Acc: 78.72% |  
LR: 0.000998
```

```
>>> Model saved! Best Acc: 78.72%
```

```
Epoch 69/200 | Loss: 0.7430 | Val Loss: 1.0611 | Val Acc: 74.47% |  
LR: 0.000997
```

```
Epoch 70/200 | Loss: 0.8186 | Val Loss: 1.2438 | Val Acc: 73.05% |  
LR: 0.000995
```

```
Epoch 71/200 | Loss: 0.7268 | Val Loss: 1.1564 | Val Acc: 74.47% |  
LR: 0.000994
```

```
Epoch 72/200 | Loss: 0.8137 | Val Loss: 1.0114 | Val Acc: 80.14% |  
LR: 0.000992
```

```
>>> Model saved! Best Acc: 80.14%
```

```
Epoch 73/200 | Loss: 0.7535 | Val Loss: 1.0206 | Val Acc: 76.60% |  
LR: 0.000990
```

```
Epoch 74/200 | Loss: 0.7388 | Val Loss: 1.1019 | Val Acc: 75.18% |  
LR: 0.000988
```

```
Epoch 75/200 | Loss: 0.7445 | Val Loss: 1.1318 | Val Acc: 74.47% |  
LR: 0.000985
```

```
Epoch 76/200 | Loss: 0.6937 | Val Loss: 1.1448 | Val Acc: 75.89% |  
LR: 0.000982
```

```
Epoch 77/200 | Loss: 0.6918 | Val Loss: 1.1467 | Val Acc: 73.05% |  
LR: 0.000980
```

```
Epoch 78/200 | Loss: 0.6793 | Val Loss: 1.1574 | Val Acc: 73.05% |  
LR: 0.000976
```

```
Epoch 79/200 | Loss: 0.7050 | Val Loss: 1.1034 | Val Acc: 75.89% |  
LR: 0.000973
```

```
Epoch 80/200 | Loss: 0.6415 | Val Loss: 1.0612 | Val Acc: 74.47% |  
LR: 0.000970
```

```
Epoch 81/200 | Loss: 0.5909 | Val Loss: 1.0726 | Val Acc: 78.01% |  
LR: 0.000966
```

```
Epoch 82/200 | Loss: 0.6076 | Val Loss: 1.1640 | Val Acc: 75.18% |  
LR: 0.000962
```

```
Epoch 83/200 | Loss: 0.6668 | Val Loss: 1.2546 | Val Acc: 70.21% |  
LR: 0.000958
```

```
Epoch 84/200 | Loss: 0.6556 | Val Loss: 1.4205 | Val Acc: 72.34% |  
LR: 0.000953
```

```
Epoch 85/200 | Loss: 0.6958 | Val Loss: 1.2075 | Val Acc: 73.05% |  
LR: 0.000949
```

```
Epoch 86/200 | Loss: 0.6594 | Val Loss: 1.2367 | Val Acc: 75.89% |  
LR: 0.000944
```

```
Epoch 87/200 | Loss: 0.6188 | Val Loss: 1.1370 | Val Acc: 74.47% |  
LR: 0.000939
```

```
Epoch 88/200 | Loss: 0.6060 | Val Loss: 1.1203 | Val Acc: 76.60% |  
LR: 0.000934
```

```
Epoch 89/200 | Loss: 0.6046 | Val Loss: 1.1968 | Val Acc: 73.05% |  
LR: 0.000929
```

```
Epoch 90/200 | Loss: 0.5798 | Val Loss: 1.4020 | Val Acc: 73.05% |  
LR: 0.000923
```

```
Epoch 91/200 | Loss: 0.6169 | Val Loss: 1.1847 | Val Acc: 75.89% |  
LR: 0.000917
```

```
Epoch 92/200 | Loss: 0.6175 | Val Loss: 1.2837 | Val Acc: 71.63% |  
LR: 0.000911
```

```
Epoch 93/200 | Loss: 0.5954 | Val Loss: 1.2089 | Val Acc: 75.18% |
```

```
LR: 0.000905
Epoch 94/200 | Loss: 0.5736 | Val Loss: 1.2248 | Val Acc: 75.18% |
LR: 0.000899
Epoch 95/200 | Loss: 0.5603 | Val Loss: 1.2408 | Val Acc: 74.47% |
LR: 0.000892
Epoch 96/200 | Loss: 0.5870 | Val Loss: 1.2201 | Val Acc: 74.47% |
LR: 0.000886
Epoch 97/200 | Loss: 0.5695 | Val Loss: 1.2530 | Val Acc: 75.89% |
LR: 0.000879
Epoch 98/200 | Loss: 0.5759 | Val Loss: 1.2197 | Val Acc: 73.76% |
LR: 0.000872
Epoch 99/200 | Loss: 0.5271 | Val Loss: 1.1985 | Val Acc: 75.18% |
LR: 0.000865
Epoch 100/200 | Loss: 0.5522 | Val Loss: 1.2121 | Val Acc: 75.89% |
LR: 0.000857
Epoch 101/200 | Loss: 0.5600 | Val Loss: 1.1863 | Val Acc: 78.01% |
LR: 0.000850
Epoch 102/200 | Loss: 0.5817 | Val Loss: 1.1488 | Val Acc: 78.72% |
LR: 0.000842
Epoch 103/200 | Loss: 0.5522 | Val Loss: 1.0599 | Val Acc: 80.14% |
LR: 0.000835
Epoch 104/200 | Loss: 0.5538 | Val Loss: 1.1036 | Val Acc: 80.14% |
LR: 0.000827
Epoch 105/200 | Loss: 0.5251 | Val Loss: 1.1528 | Val Acc: 80.14% |
LR: 0.000819
Epoch 106/200 | Loss: 0.5204 | Val Loss: 1.1420 | Val Acc: 81.56% |
LR: 0.000810
    >>> Model saved! Best Acc: 81.56%
Epoch 107/200 | Loss: 0.5224 | Val Loss: 1.0714 | Val Acc: 80.85% |
LR: 0.000802
Epoch 108/200 | Loss: 0.5214 | Val Loss: 1.1869 | Val Acc: 78.01% |
LR: 0.000793
Epoch 109/200 | Loss: 0.5280 | Val Loss: 1.1515 | Val Acc: 78.01% |
LR: 0.000785
Epoch 110/200 | Loss: 0.5216 | Val Loss: 1.2276 | Val Acc: 78.01% |
LR: 0.000776
Epoch 111/200 | Loss: 0.5428 | Val Loss: 1.2287 | Val Acc: 77.30% |
LR: 0.000767
Epoch 112/200 | Loss: 0.5169 | Val Loss: 1.0685 | Val Acc: 78.01% |
LR: 0.000758
Epoch 113/200 | Loss: 0.5379 | Val Loss: 1.1293 | Val Acc: 77.30% |
LR: 0.000749
Epoch 114/200 | Loss: 0.5481 | Val Loss: 1.1975 | Val Acc: 76.60% |
LR: 0.000740
Epoch 115/200 | Loss: 0.5224 | Val Loss: 1.1025 | Val Acc: 77.30% |
LR: 0.000730
Epoch 116/200 | Loss: 0.5185 | Val Loss: 1.1594 | Val Acc: 76.60% |
LR: 0.000721
Epoch 117/200 | Loss: 0.5273 | Val Loss: 1.1519 | Val Acc: 78.01% |
LR: 0.000711
Epoch 118/200 | Loss: 0.5232 | Val Loss: 1.1587 | Val Acc: 78.72% |
LR: 0.000702
Epoch 119/200 | Loss: 0.5119 | Val Loss: 1.2454 | Val Acc: 76.60% |
```

```
LR: 0.000692
Epoch 120/200 | Loss: 0.5181 | Val Loss: 1.1174 | Val Acc: 77.30% |
LR: 0.000682
Epoch 121/200 | Loss: 0.5101 | Val Loss: 1.0895 | Val Acc: 78.01% |
LR: 0.000672
Epoch 122/200 | Loss: 0.5223 | Val Loss: 1.1679 | Val Acc: 78.01% |
LR: 0.000662
Epoch 123/200 | Loss: 0.5126 | Val Loss: 1.1371 | Val Acc: 78.01% |
LR: 0.000652
Epoch 124/200 | Loss: 0.5162 | Val Loss: 1.1500 | Val Acc: 78.01% |
LR: 0.000642
Epoch 125/200 | Loss: 0.5087 | Val Loss: 1.1289 | Val Acc: 77.30% |
LR: 0.000632
Epoch 126/200 | Loss: 0.5067 | Val Loss: 1.1609 | Val Acc: 78.01% |
LR: 0.000622
Epoch 127/200 | Loss: 0.5069 | Val Loss: 1.1386 | Val Acc: 78.01% |
LR: 0.000611
Epoch 128/200 | Loss: 0.5047 | Val Loss: 1.1309 | Val Acc: 80.14% |
LR: 0.000601
Epoch 129/200 | Loss: 0.5022 | Val Loss: 1.1210 | Val Acc: 79.43% |
LR: 0.000591
Epoch 130/200 | Loss: 0.5190 | Val Loss: 1.1202 | Val Acc: 78.72% |
LR: 0.000580
Epoch 131/200 | Loss: 0.5117 | Val Loss: 1.2080 | Val Acc: 75.89% |
LR: 0.000570
Epoch 132/200 | Loss: 0.5090 | Val Loss: 1.2512 | Val Acc: 77.30% |
LR: 0.000559
Epoch 133/200 | Loss: 0.5100 | Val Loss: 1.2081 | Val Acc: 76.60% |
LR: 0.000549
Epoch 134/200 | Loss: 0.5117 | Val Loss: 1.1714 | Val Acc: 75.89% |
LR: 0.000538
Epoch 135/200 | Loss: 0.5142 | Val Loss: 1.1536 | Val Acc: 78.01% |
LR: 0.000527
Epoch 136/200 | Loss: 0.5207 | Val Loss: 1.1262 | Val Acc: 78.01% |
LR: 0.000517
Epoch 137/200 | Loss: 0.5011 | Val Loss: 1.1573 | Val Acc: 77.30% |
LR: 0.000506
Epoch 138/200 | Loss: 0.5071 | Val Loss: 1.1492 | Val Acc: 78.01% |
LR: 0.000496
Epoch 139/200 | Loss: 0.5126 | Val Loss: 1.1393 | Val Acc: 78.72% |
LR: 0.000485
Epoch 140/200 | Loss: 0.5028 | Val Loss: 1.1352 | Val Acc: 78.72% |
LR: 0.000474
Epoch 141/200 | Loss: 0.4864 | Val Loss: 1.1635 | Val Acc: 78.72% |
LR: 0.000464
Epoch 142/200 | Loss: 0.4977 | Val Loss: 1.1407 | Val Acc: 78.01% |
LR: 0.000453
Epoch 143/200 | Loss: 0.4998 | Val Loss: 1.1471 | Val Acc: 78.01% |
LR: 0.000443
Epoch 144/200 | Loss: 0.5040 | Val Loss: 1.1554 | Val Acc: 78.72% |
LR: 0.000432
Epoch 145/200 | Loss: 0.5043 | Val Loss: 1.1392 | Val Acc: 76.60% |
LR: 0.000422
```

Epoch 146/200 | Loss: 0.5050 | Val Loss: 1.1065 | Val Acc: 79.43% |
LR: 0.000411

Early stopping triggered after 146 epochs (patience: 40)
Best Validation Accuracy: 81.56%

Training completed in 41.32 minutes

```

In [105]: # --- Final Evaluation with Classification Report and Confusion Matrix ---
from sklearn.metrics import classification_report, accuracy_score

# Plot training history
plt.figure(figsize=(14, 4))

plt.subplot(1, 3, 1)
plt.plot(training_history['loss'], label='Training Loss')
plt.plot(training_history['val_loss'], label='Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Training vs Validation Loss')
plt.legend()
plt.grid(True, alpha=0.3)

plt.subplot(1, 3, 2)
plt.plot(training_history['val_acc'], label='Validation Accuracy')
plt.axhline(y=best_acc, color='r', linestyle='--', label=f'Best: {best_acc*100:.2f}%')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.title('Validation Accuracy')
plt.legend()
plt.grid(True, alpha=0.3)

plt.subplot(1, 3, 3)
plt.scatter(training_history['val_loss'], training_history['val_acc'])
plt.xlabel('Validation Loss')
plt.ylabel('Validation Accuracy')
plt.title('Loss vs Accuracy Trade-off')
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('training_history.png', dpi=150, bbox_inches='tight')
plt.show()
print("Training history plot saved as 'training_history.png'")

# Load best model for final evaluation
model.load_state_dict(torch.load('landmark_facial_expression_model.pth'))
model.eval()
with torch.no_grad():
    test_outputs = model(X_test_norm, test_lengths)
    test_pred = test_outputs.argmax(dim=1).cpu().numpy()
    test_true = y_test.cpu().numpy()
    test_probabilities = torch.softmax(test_outputs, dim=1).cpu().numpy()

# Accuracy
acc = accuracy_score(test_true, test_pred)
print(f"\n{'='*60}")
print(f"FINAL TEST ACCURACY: {acc * 100:.2f}%")
print(f"{'='*60}")

# Per-class accuracy

```

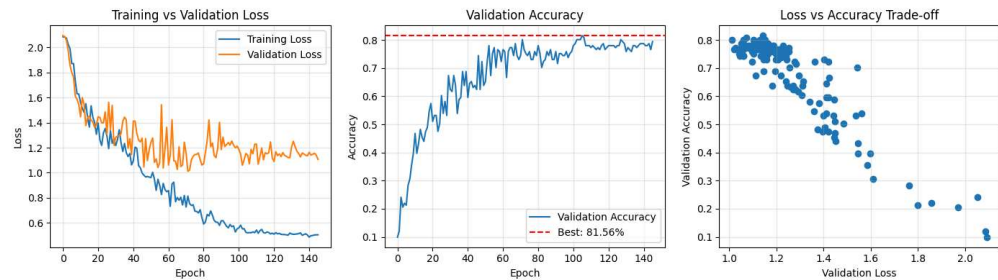


```

print("\nPer-Class Accuracy:")
for i, label in enumerate(labels):
    mask = test_true == i
    if mask.sum() > 0:
        class_acc = (test_pred[mask] == i).sum() / mask.sum()
        class_count = mask.sum()
        print(f" {label:<12} : {class_acc*100:6.2f}% ({class_count}
samples)")

# Classification Report
print("\nDetailed Classification Report:")
print(classification_report(test_true, test_pred, target_names=labels,
digits=4))

```



Training history plot saved as 'training_history.png'

=====

FINAL TEST ACCURACY: 81.56%

=====

Per-Class Accuracy:

```

Angry      : 78.57% (28 samples)
Calm       : 92.86% (14 samples)
Disgust    : 95.65% (23 samples)
Fearful    : 77.27% (22 samples)
Happy      : 93.75% (16 samples)
Neutral    : 75.00% (8 samples)
Sad        : 64.71% (17 samples)
Surprised  : 69.23% (13 samples)

```

Detailed Classification Report:

	precision	recall	f1-score	support
Angry	0.9565	0.7857	0.8627	28
Calm	0.9286	0.9286	0.9286	14
Disgust	0.8462	0.9565	0.8980	23
Fearful	0.7727	0.7727	0.7727	22
Happy	0.9375	0.9375	0.9375	16
Neutral	0.7500	0.7500	0.7500	8
Sad	0.7857	0.6471	0.7097	17
Surprised	0.5000	0.6923	0.5806	13
accuracy			0.8156	141
macro avg	0.8096	0.8088	0.8050	141

weighted avg	0.8305	0.8156	0.8186	141
--------------	--------	--------	--------	-----

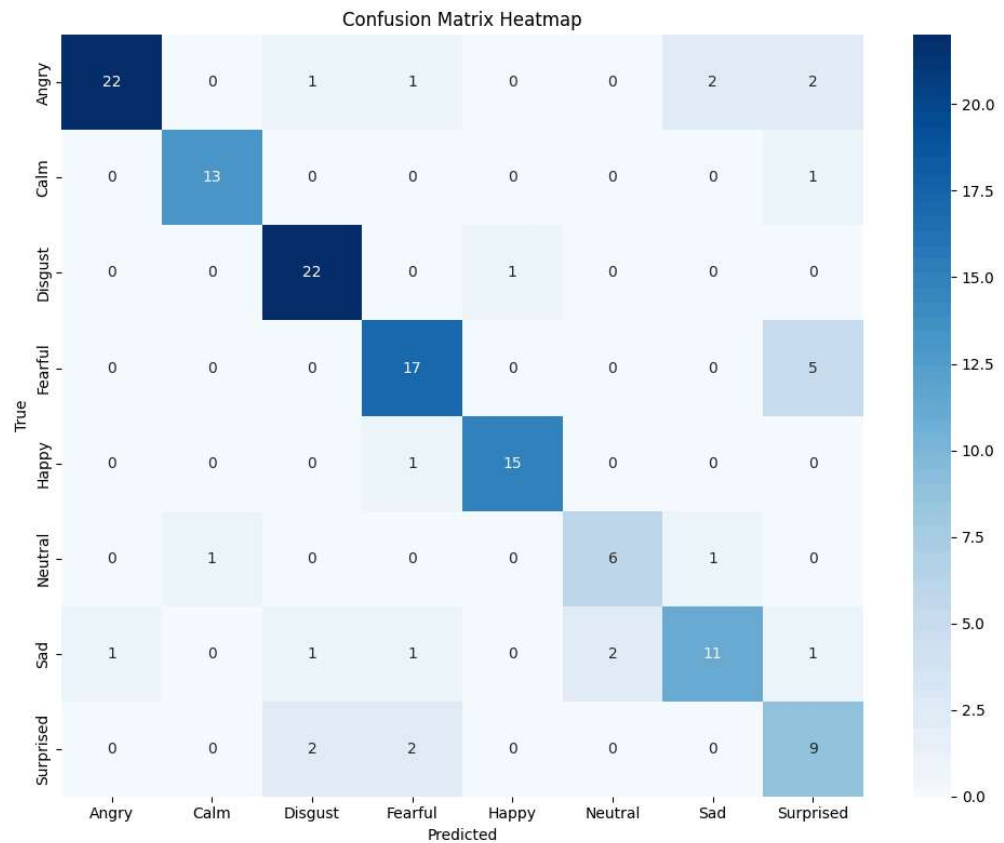
```
In [106]: # Confusion Matrix
cm = confusion_matrix(test_true, test_pred)
print("\nConfusion Matrix:")
print(cm)
```

```
Confusion Matrix:
[[22  0  1  1  0  0  2  2]
 [ 0 13  0  0  0  0  0  1]
 [ 0  0 22  0  1  0  0  0]
 [ 0  0  0 17  0  0  0  5]
 [ 0  0  0  1 15  0  0  0]
 [ 0  1  0  0  0  6  1  0]
 [ 1  0  1  1  0  2 11  1]
 [ 0  0  2  2  0  0  0  9]]
```

```

In [107]: # Heatmap
plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=labels,
            yticklabels=labels)
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title('Confusion Matrix Heatmap')
plt.tight_layout()
plt.savefig('confusion_heatmap.png')
plt.show()
print("Heatmap saved as 'confusion_heatmap.png'")

```



Heatmap saved as 'confusion_heatmap.png'

```

In # --- Test on a New Video ---
[115]: test_video_path = os.path.join('testing', 'Fearful', '01-01-06-01-01-01-13.mp4')

# Extract features from the video
print(f"Testing on: {test_video_path}")
print(f"File exists: {os.path.exists(test_video_path)}")

features_list = extract_features_from_video(test_video_path)

if not features_list or len(features_list) == 0:
    print("No facial features detected in the video.")
else:
    print(f"Extracted features from {len(features_list)} frames.")

    # Flatten each frame's features into a vector
    seq = []
    for f in features_list:
        flattened = flatten_features(f)
        if flattened is not None: # Safety check
            seq.append(flattened)

    if len(seq) == 0:
        print("No valid feature vectors after flattening.")
    else:
        print(f"Processed {len(seq)} valid feature vectors.")

        # Get max sequence length from training (assuming X_padded exists)
        max_seq_len = X_padded.shape[1]
        feature_dim = X_padded.shape[2]

        # Create padded array: (1, max_seq_len, feature_dim)
        padded_seq = np.zeros((1, max_seq_len, feature_dim),
dtype=np.float32)
        actual_length = min(len(seq), max_seq_len)
        padded_seq[0, :actual_length, :] = seq[:actual_length]

        # Convert to torch tensor
        padded_tensor = torch.tensor(padded_seq, dtype=torch.float32)

        # Normalize using the SAME scaler from training
        test_tensor = normalize_padded_sequences(padded_tensor) # This
returns a torch tensor

        # Length tensor
        test_length = torch.tensor([actual_length], dtype=torch.long)

        # Load best model and predict

model.load_state_dict(torch.load('landmark_facial_expression_model.pth'))
model.eval()

with torch.no_grad():

```

```

output = model(test_tensor, test_length)
probabilities = torch.softmax(output, dim=1)[0]
pred_idx = output.argmax(dim=1).item()
confidence = probabilities[pred_idx].item()

predicted_expression = labels[pred_idx]

print("\n" + "="*50)
print(f"PREDICTED EXPRESSION: {predicted_expression.upper()}")
print(f"Confidence: {confidence * 100:.2f}%")
print("="*50)

# Optional: Show top-3 predictions
print("\nTop 3 Predictions:")
topk_vals, topk_idx = torch.topk(probabilities, 3)
for i in range(3):
    expr = labels[topk_idx[i]]
    conf = topk_vals[i].item() * 100
    print(f" {i+1}. {expr:<10} : {conf:.2f}%")

```

Testing on: testing\Fearful\01-01-06-01-01-13.mp4
 File exists: True
 Processed 94 frames.
 Detected faces in 94 frames (100.00%).
 Extracted features from 94 frames.
 Processed 94 valid feature vectors.

```

=====
PREDICTED EXPRESSION: FEARFUL
Confidence: 83.69%
=====

```

```

Top 3 Predictions:
1. Fearful      : 83.69%
2. Sad          : 5.10%
3. Surprised    : 3.41%

```